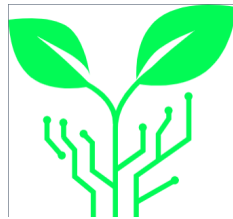


Democratic and Popular Republic of Algeria
Ministry of Higher Education and Scientific Research
Higher School of Computer and Digital Sciences and Technologies
MI department



2CP Project
“Projet Pluridisciplinaire”

Theme



Design and Implementation of AGOP

An AI-Powered Farming Mobile Application

Performed by:

Boukandoul Mahdi (*team leader*)
Aissa Kifouch
Boukhenchouch Mohyiddin

Supervised by:

M. BELLAL HAFHOUF

Academic Year 2024–2025

Abstract

Context. Agriculture represents a vital pillar of the Algerian economy, yet the majority of small and medium-scale farmers still rely on experience-based, manual decision-making without access to modern data-driven tools. The global rise of smart farming and precision agriculture — driven by mobile computing, artificial intelligence, and real-time data collection — creates a direct opportunity to deliver agricultural intelligence to farmers through affordable smartphones, which have seen rapid adoption across Algeria in recent years.

Problem. Farmers face daily decisions regarding irrigation, fertilization, and harvesting without any structured support system. Existing smart farming platforms such as Farmbrite, Agrivi, and Granular are subscription-based, designed for large-scale enterprises, and not adapted to Algerian crop types, languages, or contexts. The core gap is the absence of a free, localised, and intelligent farming assistant accessible on a standard Android smartphone, capable of delivering AI-driven irrigation recommendations grounded in real agronomic data.

Methodology. AGOP was developed as a cross-platform mobile application using **Flutter** (Dart) for the frontend and **FastAPI** (Python 3.11) for the RESTful backend, with an **SQLite** database managed through **SQLAlchemy ORM**. The AI core integrates a pre-trained **LightGBM** model loaded via **joblib** to predict crop yield under two scenarios — with and without irrigation — and derives personalised irrigation recommendations combined with live **OpenWeatherMap** data. The development followed the **V-Model** methodology, pairing each design phase with a corresponding validation step.

Results. The resulting application provides farmers with a personalised home dashboard displaying real-time weather metrics (temperature, humidity, wind, UV index, 5-day forecast), a crop management module supporting multiple crop types and soil profiles with growth-stage tracking, a task manager covering watering, fertilizing, and harvesting activities with completion tracking, and an AI-driven recommendation engine that computes yield gain in tonnes per hectare and assigns a confidence level to each irrigation decision. All 12 planned functional test cases passed successfully on Android devices, with API response times consistently below 200 ms.

Keywords: smart farming, yield prediction, irrigation optimisation, water resource management, crop monitoring, LightGBM, gradient boosting, mobile agriculture, precision agriculture, RESTful API.

Acknowledgments

We would like to express our sincere gratitude to our supervisor, **M. BELLAL HAFHOUF**, for his continuous guidance, availability, and invaluable advice throughout the development of this project. His constructive feedback and encouragement were essential to bringing AGOP to completion.

We extend our thanks to the faculty and administration of **ESTIN** — Higher School of Computer and Digital Sciences and Technologies, Béjaïa — for providing the academic environment, infrastructure, and resources that made this project possible.

We also thank our fellow students and colleagues for their support and constructive discussions during the development process.

Finally, we acknowledge the open-source community behind **Flutter**, **FastAPI**, **LightGBM**, **SQLAlchemy**, and all other libraries and tools used in this work, without which a project of this scope would not have been achievable within a single academic semester.

Contents

Abstract	i
Acknowledgments	ii
1 Introduction	1
1.1 Context	1
1.2 Problem Statement	1
1.3 Objectives of the Project	2
1.4 Report Contributions	2
1.5 Report Outline	2
2 System Analysis	4
2.1 Introduction	4
2.2 Client Presentation	4
2.3 Problem / Solution	4
2.3.1 Problem Definition	4
2.3.2 Critique of Existing Solutions	5
2.3.3 Proposed Solution	5
2.4 Requirements Analysis	5
2.4.1 Functional Requirements	5
2.4.2 Non-Functional Requirements	7
2.4.3 Use Case Descriptions	7
2.5 Conclusion	9
3 System Design	10
3.1 Introduction	10
3.2 System Architecture	10
3.2.1 Overview	10
3.2.2 Presentation Layer — Flutter Frontend	11
3.2.3 Logic Layer — FastAPI Backend	11
3.2.4 Data Layer — SQLite Database	12
3.3 UML Diagrams	12
3.3.1 Identification of Actors	12
3.3.2 Use Case Diagram	12
3.3.3 Sequence Diagrams	13
3.3.4 Class Diagram	17
3.4 Database Design	18
3.4.1 Entity-Relationship Diagram	18
3.4.2 Relational Model	19
3.4.3 Data Dictionary	20
3.5 Backend Architecture and LightGBM Integration	21
3.5.1 Three-Tier Backend Module Structure	21

3.5.2	LightGBM Integration Pipeline	21
3.6	Conclusion	22
4	Implementation, Testing & Results	23
4.1	Introduction	23
4.2	Development Details	23
4.2.1	Development Environment	23
4.2.2	Technology Choices Justification	24
4.2.3	Backend Implementation	24
4.2.4	Frontend Implementation	28
4.2.5	Application Screenshots	31
4.3	Testing, Validation and Performance	33
4.3.1	Testing Strategy	33
4.3.2	Functional Test Cases	34
4.3.3	API Validation with Postman	35
4.3.4	Performance Results	35
4.3.5	Non-Functional Requirements Compliance Summary	36
4.4	Conclusion	37
5	Conclusion and Future Work	38
5.1	Conclusion	38
5.1.1	Summary of Achievements	38
5.1.2	Lessons Learned	39
5.2	Future Work	39
5.2.1	Plant Disease Detection via Image Recognition	39
5.2.2	IoT Sensor Integration	39
5.2.3	Offline Mode	39
5.2.4	Multilingual Support (Arabic and French)	40
5.2.5	Continuous Model Retraining	40
5.2.6	Weather-Aware Notification System	40
5.2.7	Production Deployment and Scalability	40
	References	41
	Appendices	42
	A Glossary	43
	B REST API Endpoint Reference	46
	C LightGBM Model Feature Description	47
	D Project Timeline	48

Chapter 1

Introduction

1.1 Context

Agriculture is one of the most fundamental sectors of the Algerian economy, employing a large portion of the rural population and contributing significantly to national food security. Despite its importance, the sector lags behind in the adoption of modern technologies, particularly when compared to industrialised nations where precision agriculture and smart farming have already transformed how crops are managed.

Globally, the emergence of intelligent farming solutions — combining mobile applications, the Internet of Things (IoT), and artificial intelligence (AI) — has demonstrated measurable improvements in crop yields, water usage efficiency, and overall farm profitability. Smartphone penetration in Algeria has grown substantially in recent years, making mobile applications one of the most viable and cost-effective channels for delivering agricultural intelligence directly to farmers in the field.

In parallel, advances in machine learning, particularly gradient boosting algorithms such as **LightGBM** (Light Gradient Boosting Machine), have made it possible to train highly accurate predictive models on agricultural datasets and deploy them within lightweight backend services. These models can predict crop yield under different conditions and generate data-driven irrigation recommendations that would otherwise require the expertise of an agronomist.

1.2 Problem Statement

Despite the availability of global smart farming platforms, Algerian farmers face a specific and largely unaddressed challenge: the complete absence of a free, locally adapted, and intelligent farming assistant accessible on a standard Android smartphone.

The key problems are as follows:

- **Manual, intuition-based decisions:** Most farmers decide when and how much to irrigate based on habit and experience, without considering current weather forecasts, soil conditions, or crop growth stages. This leads to over-watering or under-watering, both of which reduce yield and waste resources.
- **No structured task management:** There is no system to schedule, track, or remind farmers of critical farming activities such as fertilization or harvesting, causing missed windows that directly impact productivity.
- **Inaccessibility of existing solutions:** Platforms such as Farmbrite, Agrivi, and Granular require paid subscriptions, are designed for large-scale agribusinesses, and

offer no localisation for Algerian crop types, climate conditions, or user context.

- **Absence of AI-powered irrigation advice:** No existing free tool combines real-time weather data with a machine learning model to generate personalised, yield-based irrigation recommendations for individual farmers.

1.3 Objectives of the Project

The primary objective of this project is to design and implement **AGOP**, a free, mobile-first, AI-powered farming application that addresses the above problems. The specific objectives are:

1. Develop a cross-platform mobile application (Android) with an intuitive interface accessible to non-technical farmers.
2. Integrate a **LightGBM** machine learning model to predict crop yield under irrigated and non-irrigated conditions, and generate personalised irrigation recommendations with confidence levels.
3. Provide GPS-based real-time weather data (temperature, humidity, wind, UV index, rainfall forecasts) directly on the farmer’s dashboard.
4. Enable the registration and tracking of multiple crops across different fields, with soil type, area, planting date, and growth-stage monitoring.
5. Implement a task management system for watering, fertilizing, and harvesting activities, with due dates and completion tracking.
6. Build a secure, scalable RESTful backend API using **FastAPI** and **SQLAlchemy ORM** to support all mobile operations with response times below 200 ms.

1.4 Report Contributions

This report documents the complete development lifecycle of AGOP, from initial requirements analysis to system design, implementation, and functional testing. The specific contributions of this report are:

- A thorough market study and requirements analysis comparing existing smart farming solutions and identifying the unmet needs of Algerian farmers.
- A complete system design including a three-tier architecture, UML diagrams (use case, sequence, class), an Entity-Relationship database diagram, and a detailed backend module description.
- A documented integration of a pre-trained **LightGBM** model within a FastAPI service, including the feature engineering pipeline, the dual-scenario yield prediction logic, and the confidence-based irrigation recommendation engine.
- A fully implemented and tested mobile application (Flutter) and REST API (FastAPI/Python), validated through 12 functional test cases on Android devices.
- Performance benchmarks demonstrating that all API endpoints respond within the defined non-functional requirements.

1.5 Report Outline

This report is organised as follows.

Chapter 2 describes the preliminary study and analysis of needs. It presents the project context, analyses three existing smart farming platforms (Farmbrite, Agrivi, and Granular), identifies their limitations, and defines the functional and non-functional requirements of AGOP, along with the development methodology adopted by the team.

Chapter 3 focuses on the modelling of the system components. It presents the system architecture, the identification of actors, and all UML diagrams including the use case diagram, three sequence diagrams, and the class diagram. It also details the database design through an Entity-Relationship diagram, a relational model, and a data dictionary, and closes with a description of the three-tier backend architecture and the LightGBM integration pipeline.

Chapter 4 presents the implementation of AGOP. It covers the development environment and tools, the backend project structure, the key REST API endpoints, sample request payloads, and the principal interfaces of the mobile application illustrated through annotated screenshots. It concludes with functional test results and performance measurements.

Finally, **Chapter 5** concludes the report with a synthesis of the achievements of the AGOP project and discusses possible future directions, including plant disease detection via image recognition, IoT sensor integration, offline mode, and multilingual support.

Chapter 2

System Analysis

2.1 Introduction

This chapter analyses the AGOP system from a requirements perspective. It begins by identifying the target users, then defines the core problem and the proposed solution, including a critique of existing platforms. It concludes with a detailed requirements specification — both functional and non-functional — and a set of use case descriptions that formalise the key interactions between the farmer and the system.

2.2 Client Presentation

The primary client of AGOP is the **Algerian farmer** — an individual responsible for managing one or more agricultural fields at small to medium scale. This user profile is characterised by the following traits:

- Manages multiple crop types (wheat, tomato, corn, etc.) across different fields with varying soil compositions and surface areas.
- Makes daily decisions regarding irrigation, fertilization, and harvesting, currently based on experience rather than data.
- Owns a smartphone (Android) but is not a technical expert — the interface must be simple, visual, and require no prior training.
- Has limited or intermittent internet connectivity in rural areas.
- Cannot afford expensive subscription-based agricultural platforms.

A secondary client is the **AI recommendation engine** acting as an internal system actor: it consumes crop data and weather inputs to autonomously generate irrigation advice and farming tasks without requiring manual input from the farmer.

2.3 Problem / Solution

2.3.1 Problem Definition

Algerian farmers currently operate without any structured, data-driven support system for their daily farming decisions. The consequences are direct and measurable:

- **Inefficient irrigation:** Without weather-aware guidance, farmers over-water or under-water their crops, wasting water and reducing yield.
- **No activity tracking:** Watering and fertilization dates are not recorded, making it impossible to evaluate past decisions or plan future ones.

- **Missed critical windows:** Fertilization and harvesting deadlines are missed due to the absence of a reminder or task system.
- **No yield prediction:** Farmers have no tool to estimate the impact of irrigation on their final harvest in tonnes per hectare.

2.3.2 Critique of Existing Solutions

Three leading smart farming platforms were analysed and compared:

- **Farmbrite:** Comprehensive farm management with crop tracking and task management, but requires a paid subscription (\$29+/month), offers no AI irrigation advice, and is not adapted to Algerian crop types.
- **Agrivi:** Includes weather integration and agronomic recommendations, but targets professional agronomists with a complex interface, enterprise-level pricing, and no Arabic or French support.
- **Granular (Corteva):** Designed exclusively for large agribusinesses, with field mapping and workforce management tools that are irrelevant and inaccessible to individual small farmers.

The shared limitations of all three platforms are: high cost, complexity, no free AI-driven irrigation advice, and no localisation for the Algerian agricultural context.

2.3.3 Proposed Solution

AGOP proposes a **free, lightweight, and intuitive mobile application** that addresses each identified gap:

- Real-time, GPS-based weather data (temperature, humidity, wind, UV index, 5-day forecast, sunrise/sunset) displayed on a personalised home dashboard.
- Crop registration with type, field name, soil type, area (ha), and planting date, with automatic growth-stage tracking.
- Daily activity tracking showing days since last watering and fertilization per crop.
- Task management system for Watering, Fertilizing, and Harvesting, with due dates and completion timestamps.
- An AI-powered irrigation recommendation engine using a pre-trained **LightGBM** model that predicts yield gain in t/ha and assigns a confidence level (high / moderate / low) to each recommendation, informed by live OpenWeatherMap data.

2.4 Requirements Analysis

2.4.1 Functional Requirements

The following table lists what the AGOP system must do:

Table 2.1: Functional Requirements of AGOP

ID	Description
FR-01	User can create an account by entering email, username, and password.

(continued)

ID	Description
FR-02	User can log in using username or email and password.
FR-03	User can view a personalised home dashboard showing their name and a greeting.
FR-04	System displays real-time GPS-based weather: temperature, humidity, rain, wind speed, and UV index.
FR-05	System displays a 5-day weather forecast with daily temperature and weather icons.
FR-06	System displays sunrise time, total daylight duration, and sunset time.
FR-07	User can add a new crop by selecting crop type, entering field name, soil type, area (ha), and planting date.
FR-08	System displays crop statistics: number of active crops, total hectares, and crops needing action.
FR-09	Each crop card displays current growth stage, stage progress bar, days since last watered, days since last fertilized, and field area.
FR-10	User can delete an existing crop from the system.
FR-11	User can view all farming tasks for the current day with a completion progress counter.
FR-12	User can create a new task by selecting task type (Watering / Fertilizing / Harvesting), associated crop, and due date.
FR-13	User can mark a task as completed; the system records the completion timestamp and moves the task to the Completed section.
FR-14	System records daily farming logs (water quantity in litres, fertilizer quantity in kg) per crop.
FR-15	System generates AI-driven irrigation recommendations using the LightGBM model, returning recommended action, yield gain (t/ha), and confidence level.
FR-16	System auto-generates farming tasks based on crop growth stage and weather conditions via the backend AI service.

2.4.2 Non-Functional Requirements

Table 2.2: Non-Functional Requirements of AGOP

ID	Category	Requirement
NFR-01	Performance	The home screen must load in under 3 seconds on a standard 4G connection.
NFR-02	Performance	All API endpoints must respond in under 200 ms excluding external weather API calls.
NFR-03	Security	User passwords must be hashed with SHA-256 before storage; no plain-text credentials may be retained.
NFR-04	Usability	The interface must be navigable by a non-technical farmer without prior training or documentation.
NFR-05	Reliability	The backend API must maintain 99% uptime during active usage periods.
NFR-06	Scalability	The architecture must support the addition of new crop types, soil types, and recommendation rules without redesigning the codebase.
NFR-07	Compatibility	The mobile application must run on Android devices with API level 21 (Android 5.0) or higher.
NFR-08	Maintainability	The backend must follow a modular architecture separating routes, CRUD operations, models, schemas, and services into independent layers.
NFR-09	Data Integrity	All database records must enforce referential integrity through foreign key constraints managed by SQLAlchemy ORM.

2.4.3 Use Case Descriptions

UC-01: User Registration

Actor	Farmer (new user)
Precondition	The user does not have an existing AGOP account.
Steps	1. User opens the app and taps <i>Sign Up</i>. 2. User enters email, username, password, and confirms the password. 3. User taps <i>Create Account</i>. 4. System validates input and checks for duplicate email or username. 5. System stores the user with a SHA-256 hashed password. 6. App navigates the user to the Login screen.
Postcondition	A new farmer account is created and stored in the database.

UC-02: User Login

Actor	Registered farmer
Precondition	The user has a valid AGOP account.
Steps	1. User enters username or email and password. 2. User taps <i>Log In</i>. 3. System verifies credentials against the database. 4. App navigates to the Home dashboard.
Postcondition	User is authenticated and has access to all application features.

UC-03: Add a Crop

Actor	Authenticated farmer
Precondition	User is logged in.
Steps	1. User navigates to the <i>Crops</i> screen and taps +. 2. User selects crop type (e.g., Wheat, Tomato) from a dropdown. 3. User enters field name, selects soil type (e.g., Clay), enters area in hectares, and sets planting date. 4. User taps <i>Add Crop</i>. 5. System saves the crop and links it to the farmer's account. 6. Crop card appears in the list with a growth stage tracker.
Postcondition	New crop is stored and visible in the farmer's crop list.

UC-04: Create a Farming Task

Actor	Authenticated farmer
Precondition	At least one crop is registered.
Steps	1. User navigates to the <i>Tasks</i> screen and taps +. 2. User selects task type: <i>Watering</i>, <i>Fertilizing</i>, or <i>Harvesting</i>. 3. User selects the target crop from the dropdown. 4. User sets a due date. 5. User taps <i>Add task</i>. 6. System saves the task and displays it in the task list.
Postcondition	Task is visible in the daily task list and the home dashboard.

UC-05: Complete a Task

Actor	Authenticated farmer
Precondition	At least one pending task exists.
Steps	1. User views the task list for the current day. 2. User taps the checkbox next to the pending task. 3. System marks the task as done and records the completion timestamp. 4. Task moves to the <i>Completed</i> section. 5. Progress counter updates (e.g., “1 of 1 done, 0 remaining”).
Postcondition	Task is marked completed; the crop’s last-watered or last-fertilized date is updated accordingly.

UC-06: Get AI Irrigation Recommendation

Actor	Authenticated farmer + AI System
Precondition	At least one crop is registered; device has GPS and internet access.
Steps	1. System retrieves crop data (type, soil, area, growth stage) from the database. 2. System fetches current weather and 48h precipitation forecast from OpenWeatherMap API. 3. LightGBM model predicts yield <i>with</i> irrigation and <i>without</i> irrigation. 4. System computes yield gain in t/ha and determines confidence level. 5. If forecasted rain > 15mm in 48h, recommendation is “Do not irrigate”. 6. System stores the recommendation and returns it to the mobile app. 7. Farmer views recommended action, yield gain, and confidence level on screen.
Postcondition	Irrigation recommendation is stored and displayed to the farmer.

2.5 Conclusion

This chapter has established a clear picture of who uses AGOP and what the system must do. The client profile defined the target user as a small-scale Algerian farmer requiring a simple, free, and intelligent mobile tool. The problem analysis exposed the limitations of existing platforms and justified the proposed solution. The requirements specification formalised 16 functional requirements and 9 non-functional constraints, while the six use case descriptions provided concrete interaction scenarios covering registration, authentication, crop management, task tracking, and AI-driven irrigation advice. These requirements form the foundation for the system design presented in the following chapter.

Chapter 3

System Design

3.1 Introduction

This chapter translates the requirements identified in Chapter 2 into a concrete system design for AGOP. Starting from the high-level architecture that connects the mobile frontend to the AI-powered backend, the chapter progressively refines the design through a set of UML models: a Use Case diagram that formalises the interactions between the farmer and the system, three Sequence diagrams that trace the step-by-step message flow for the most critical operations, a Class diagram that captures the static structure of the domain, and an Entity-Relationship diagram that governs persistent storage. The chapter closes with a detailed description of the three-tier backend architecture and the LightGBM integration pipeline.

3.2 System Architecture

3.2.1 Overview

AGOP follows a classical **three-tier client-server architecture**, separating concerns across a presentation layer (mobile frontend), a logic layer (REST API backend), and a data layer (relational database). This separation ensures that each tier can evolve independently: the Flutter frontend can be updated without touching the FastAPI backend, and the database schema can be migrated without rewriting any mobile code.

Figure 3.1 illustrates the overall flow:

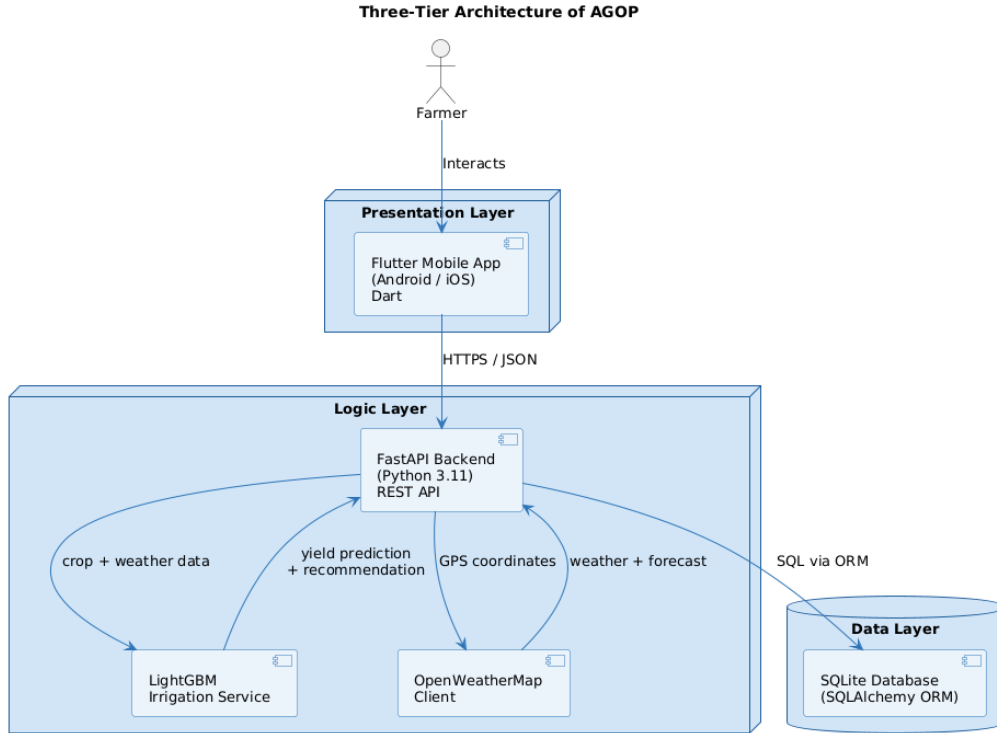


Figure 3.1: Three-tier architecture of AGOP

3.2.2 Presentation Layer — Flutter Frontend

The frontend is built with **Flutter** (Dart), enabling a single codebase to target both Android and iOS. It is responsible for:

- Rendering all user interface screens (login, dashboard, crops, tasks, recommendations).
- Capturing GPS coordinates and forwarding them to the backend for weather data retrieval.
- Sending HTTP requests to the FastAPI backend and displaying the JSON responses.
- Maintaining local session state (farmer identity, active crop selection) during the application lifecycle.

The four main screens of the application are the *Home Dashboard*, the *Crops* screen, the *Tasks* screen, and the *AI Recommendations* screen, accessed via a bottom navigation bar.

3.2.3 Logic Layer — FastAPI Backend

The backend is built with **FastAPI** (Python 3.11) and exposes a RESTful API that the mobile client consumes over HTTPS. Its responsibilities include:

- Authenticating farmers and managing their accounts.
- Performing all CRUD operations on crops, tasks, farming logs, and recommendations via **SQLAlchemy ORM**.
- Orchestrating external calls to the **OpenWeatherMap API** to retrieve real-time weather and 5-day forecasts.

- Loading the pre-trained **LightGBM** model at startup and invoking it to generate irrigation predictions and confidence-based recommendations.
- Enforcing business rules (e.g., “do not irrigate if rain > 15 mm in 48 h”) before returning a recommendation.

The backend is modular, separating `routes/`, `crud/`, `models/`, `schemas/`, and `services/` into independent layers (see Section 3.5).

3.2.4 Data Layer — SQLite Database

Persistent data is stored in an **SQLite** file managed entirely through **SQLAlchemy ORM**. SQLite was chosen for its simplicity, zero-configuration deployment, and suitability for single-server prototypes. The schema enforces referential integrity through foreign key constraints and supports seamless migration to PostgreSQL for a production deployment without changing application code.

3.3 UML Diagrams

3.3.1 Identification of Actors

Two actors interact with AGOP:

- **Farmer (primary actor):** A registered user who interacts with the mobile application to manage crops, schedule tasks, and consult irrigation recommendations.
- **AI System (secondary actor):** An internal automated component that consumes crop and weather data to produce irrigation recommendations and auto-generate farming tasks without manual input from the farmer.

3.3.2 Use Case Diagram

The Use Case diagram (Figure 3.2) maps each actor to the functional requirements defined in Chapter 2.

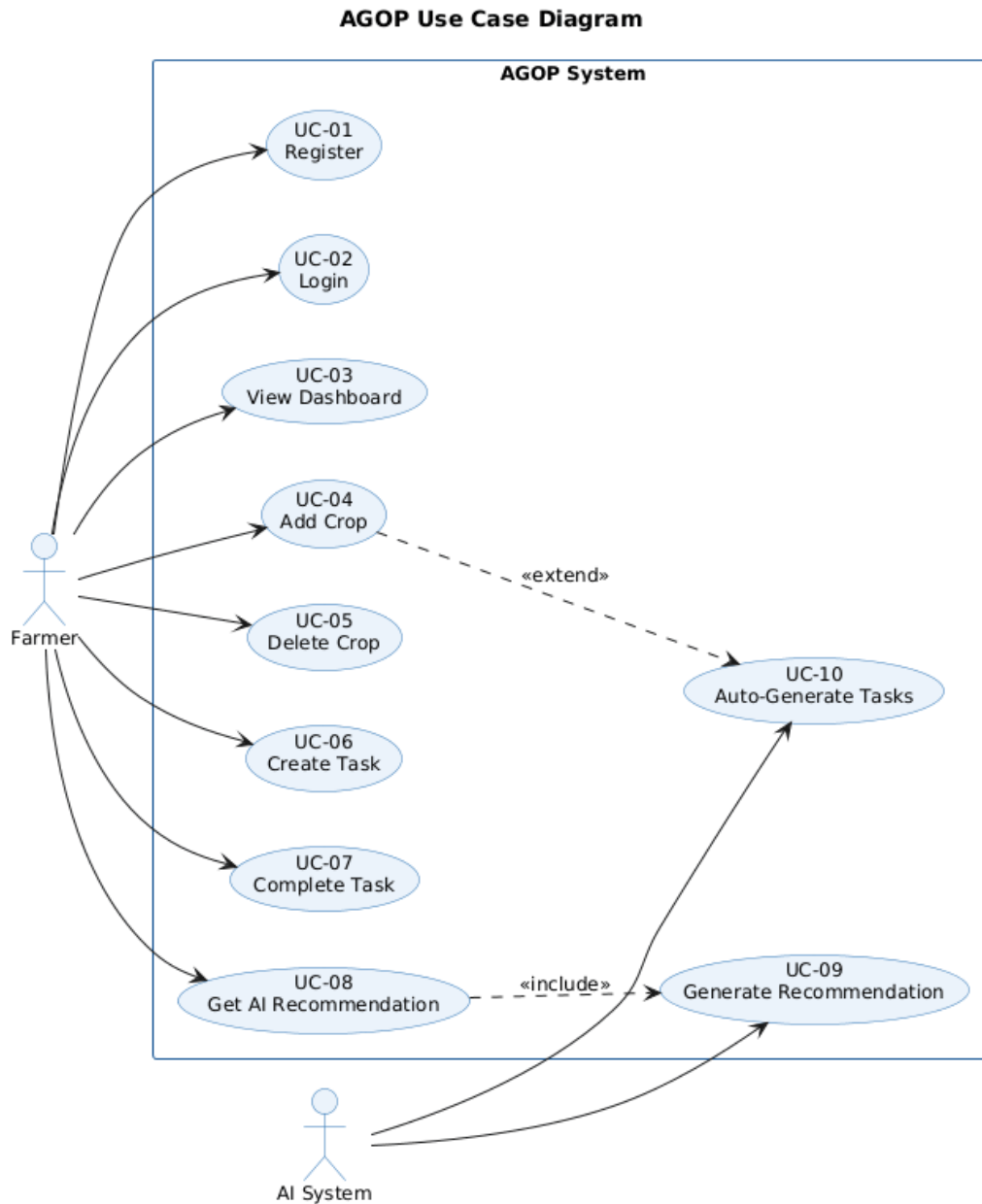


Figure 3.2: AGOP Use Case Diagram

3.3.3 Sequence Diagrams

Three sequence diagrams are presented below, covering the most critical interactions in the system.

SD-01: User Registration

Figure 3.3 traces the message flow when a new farmer creates an AGOP account.

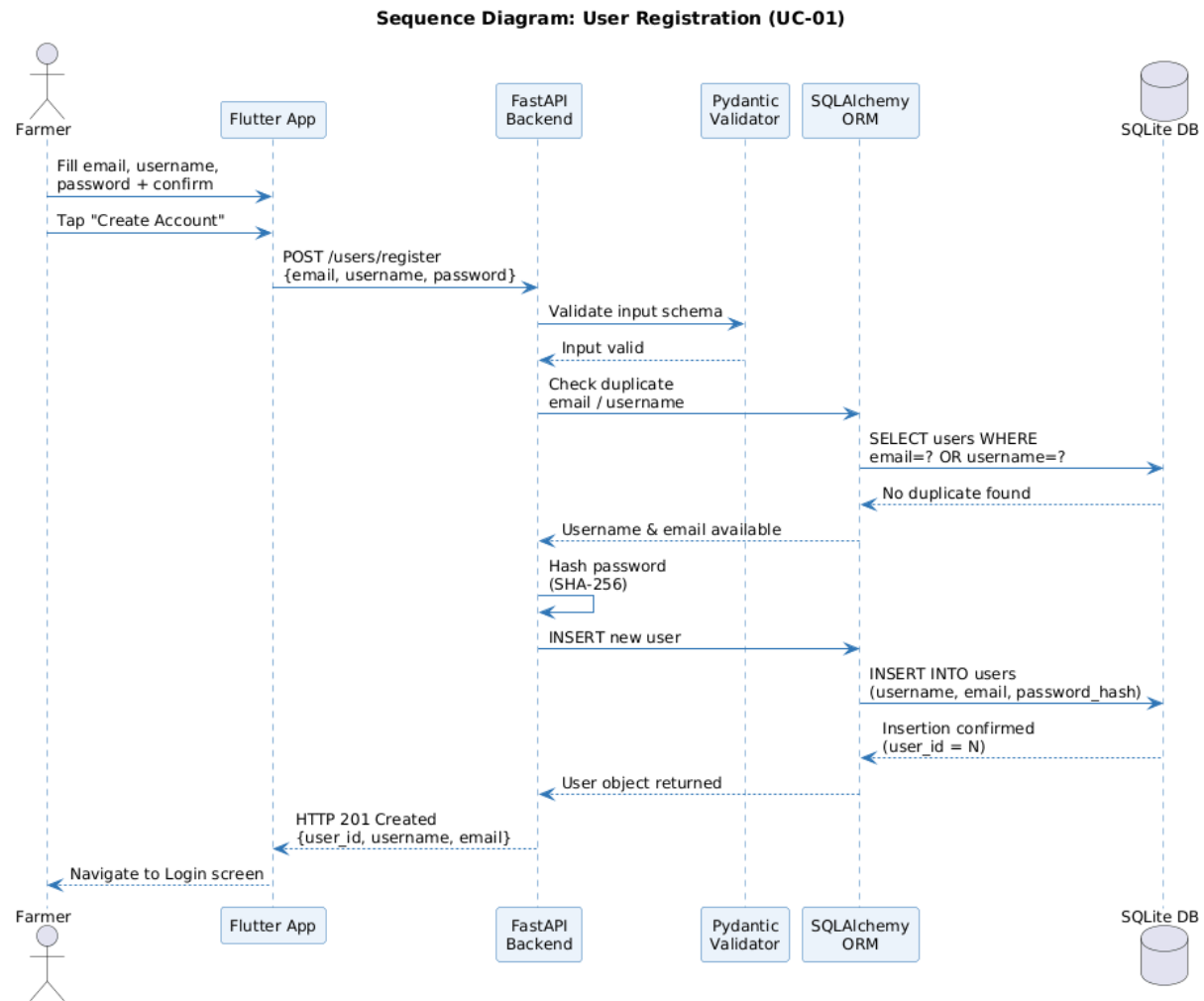


Figure 3.3: Sequence Diagram: User Registration (UC-01)

1. The farmer fills in email, username, and password on the *Sign Up* screen.
2. The Flutter frontend sends a `POST /users/register` request with the credentials as a JSON payload.
3. The FastAPI backend validates the input schema via Pydantic and checks for duplicate email/username in the database.
4. If validation passes, the backend hashes the password with SHA-256 and persists the new user record via SQLAlchemy.
5. The database confirms the insertion and returns the new user ID.
6. The backend returns `HTTP 201 Created` with the user object (excluding the hashed password).
7. The frontend navigates the farmer to the Login screen.

SD-02: Add a Crop

Figure 3.4 traces the creation of a new crop entry.

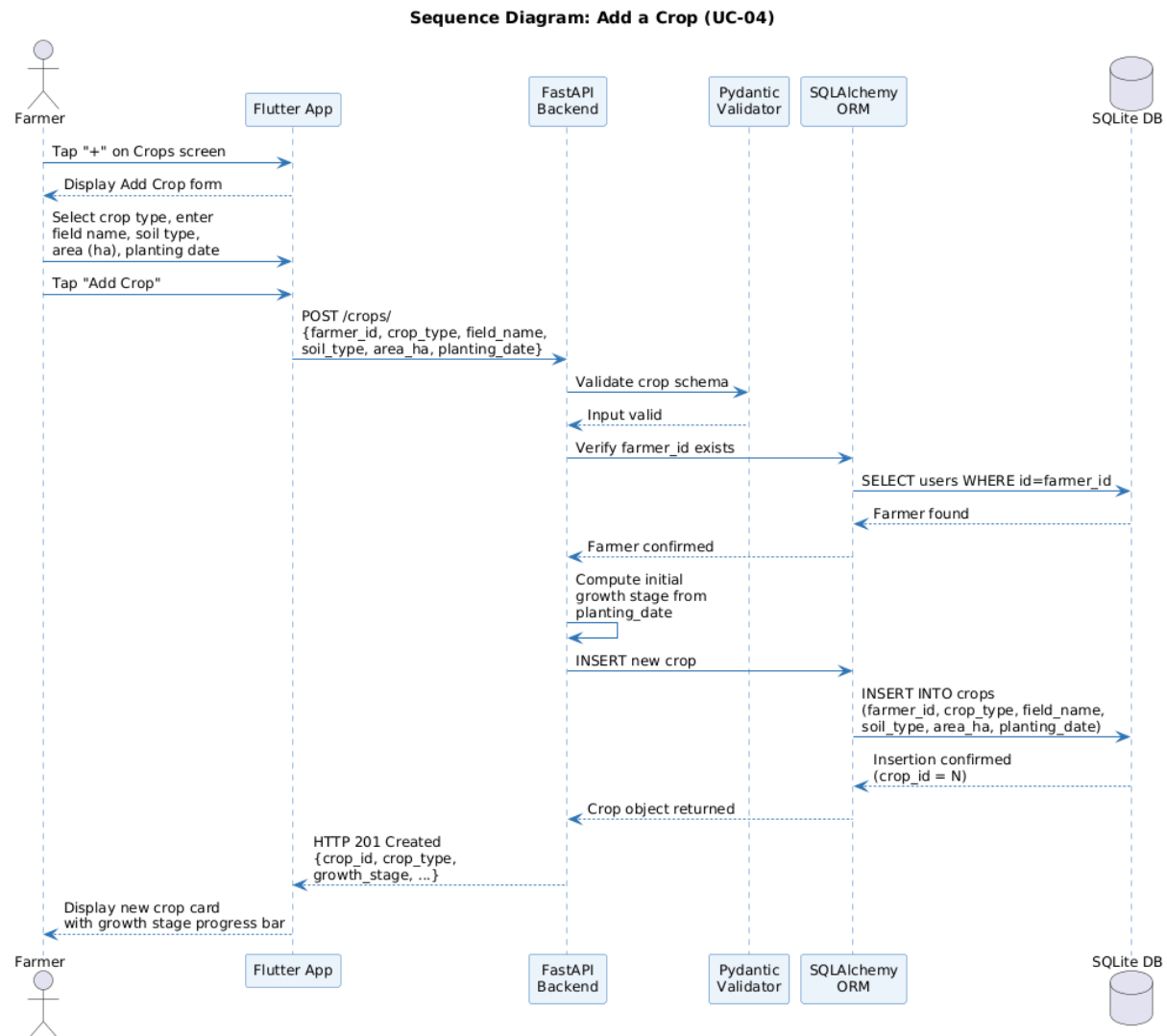


Figure 3.4: Sequence Diagram: Add a Crop (UC-04)

1. The authenticated farmer taps + on the Crops screen and fills in crop type, field name, soil type, area, and planting date.
2. The frontend sends `POST /crops/` with the crop data and the farmer's user ID.
3. The backend persists the crop via SQLAlchemy, linking it to the farmer's account through a foreign key constraint.
4. The database confirms the insertion and returns the assigned crop ID.
5. The backend returns `HTTP 201 Created` with the full crop object, including the computed initial growth stage.
6. The frontend appends the new crop card to the list and displays the growth-stage progress bar.

SD-03: Get AI Irrigation Recommendation

Figure 3.5 traces the most complex flow in the system, involving the LightGBM model and the OpenWeatherMap API.

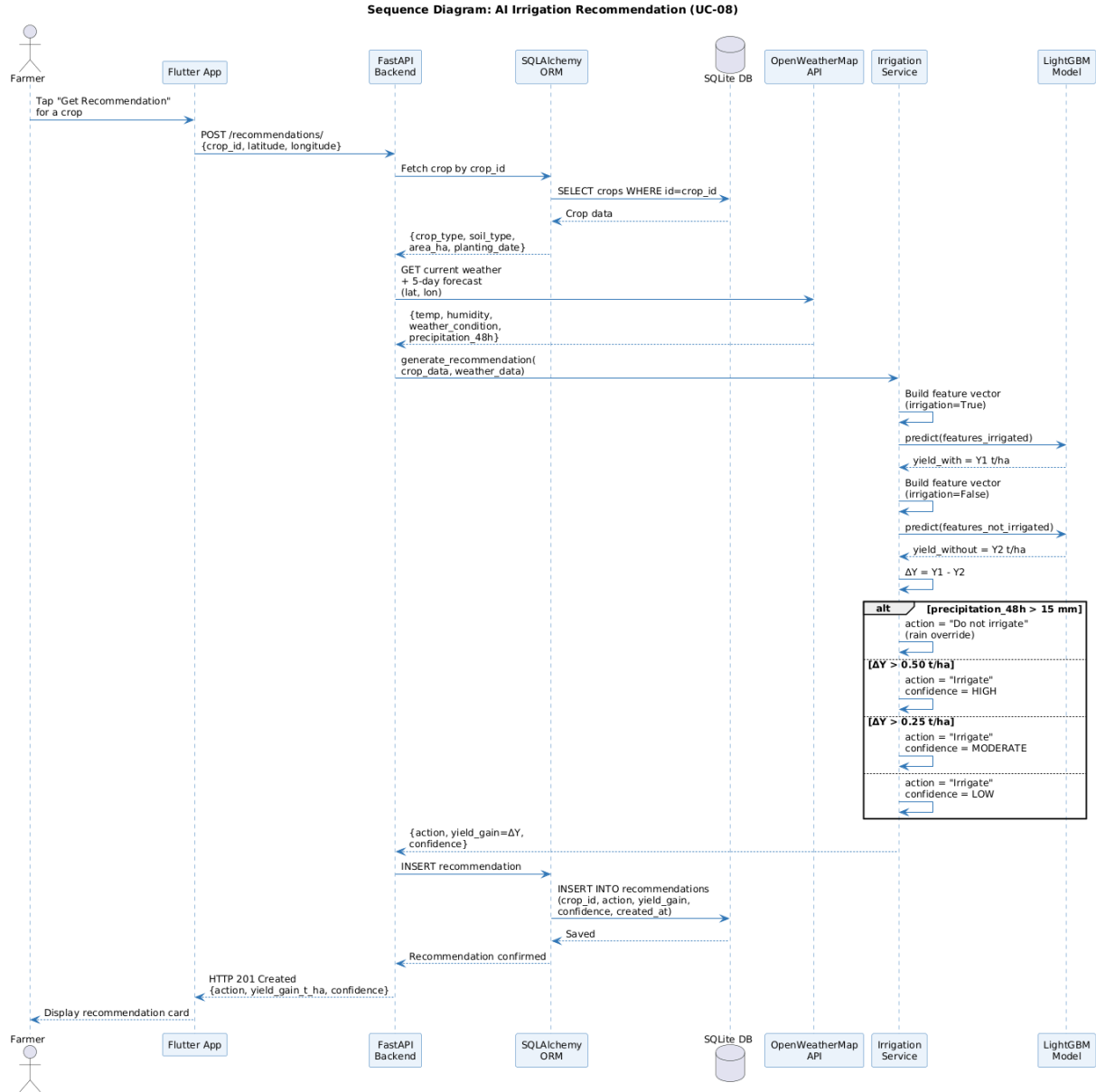


Figure 3.5: Sequence Diagram: AI Irrigation Recommendation (UC-08)

1. The farmer taps *Get Recommendation* for a specific crop on the Recommendations screen; the frontend sends GPS coordinates with `POST /recommendations/`.
2. The backend retrieves crop data (type, soil, area, growth stage) from the database via SQLAlchemy.
3. The backend queries **OpenWeatherMap** for current weather and the 48-hour precipitation forecast.
4. The irrigation service builds two feature vectors from the crop and weather data: one with `Irrigation_Used = True`, one with `Irrigation_Used = False`.
5. The pre-loaded **LightGBM** model predicts yield (t/ha) for both scenarios.
6. The service computes the yield gain, applies the rain threshold rule (>15 mm in 48 h $\Rightarrow$ “Do not irrigate”), and assigns a confidence level (high / moderate / low).
7. The backend persists the recommendation and returns the action, yield gain, and confidence level as JSON.
8. The frontend renders the recommendation card to the farmer.

3.3.4 Class Diagram

Figure 3.6 presents the class diagram for AGOP, covering the five core domain entities and their relationships.

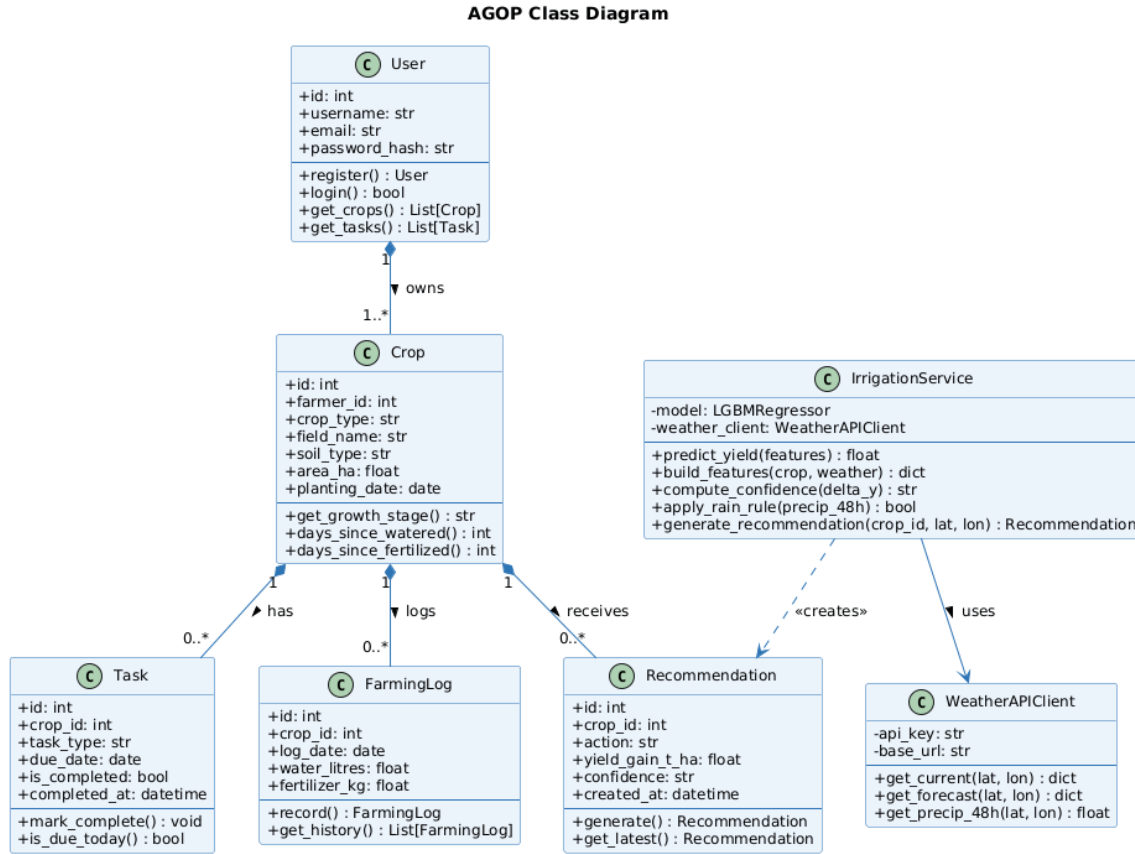


Figure 3.6: AGOP Class Diagram

Table 3.1 describes each class, its key attributes, and its principal methods.

Table 3.1: Class Descriptions

Class	Attributes	Methods / Responsibilities
User	id: int (PK) username: str email: str password_hash: str	register(), login(), get_crops(), get_tasks()
Crop	id: int (PK) farmer_id: int (FK) crop_type: str field_name: str soil_type: str area_ha: float	

(continued)

Class	Attributes	Methods / Responsibilities
Task	planting_date: date id: int (PK) crop_id: int (FK) task_type: str due_date: date is_completed: bool completed_at: datetime	get_growth_stage(), days_since_watered(), days_since_fertilized() mark_complete(), is_due_today()
FarmingLog	id: int (PK) crop_id: int (FK) log_date: date water_litres: float fertilizer_kg: float	record(), get_history()
Recommendation	id: int (PK) crop_id: int (FK) action: str yield_gain_t_ha: float confidence: str created_at: datetime	generate(), get_latest()
IrrigationService	model: LGBMRegressor weather_client: Weather-API	predict_yield(), build_features(), compute_confidence(), apply_rain_rule()

The key relationships between classes are as follows:

- **User** 1..* *owns* **Crop** (a farmer registers one or more crops).
- **Crop** 1..* *has* **Task** (each crop can have multiple scheduled tasks).
- **Crop** 1..* *logs* **FarmingLog** (each crop accumulates daily watering and fertilization records).
- **Crop** 1..* *receives* **Recommendation** (each crop can have multiple historical recommendations).
- **IrrigationService** *uses* **Recommendation** (the service populates and persists recommendation objects).

3.4 Database Design

3.4.1 Entity-Relationship Diagram

Figure 3.7 presents the Entity-Relationship diagram of the AGOP database.

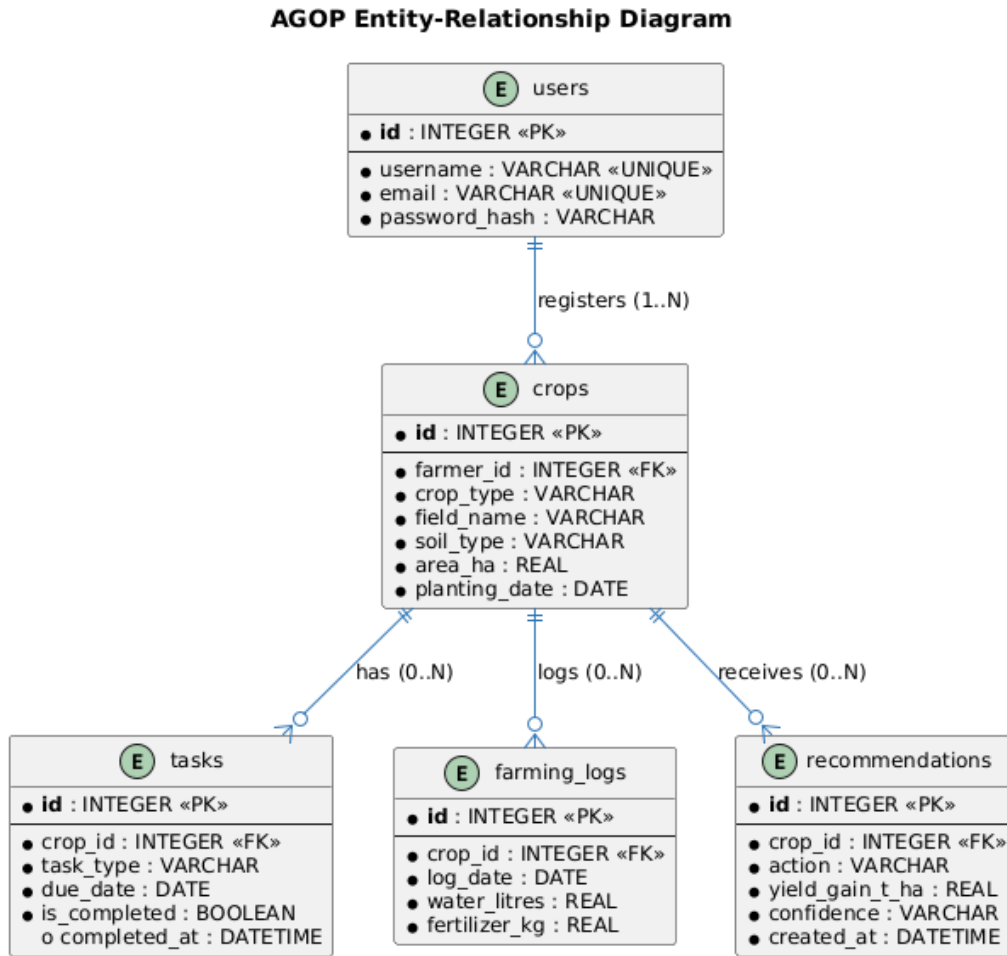


Figure 3.7: AGOP Entity-Relationship Diagram

Conseil qualité Sur PlantText, avant de télécharger, clique sur "PNG" (pas SVG). Si le diagramme est trop petit, ajoute cette ligne juste après @startuml dans chaque code :

skinparam dpi 150

3.4.2 Relational Model

The ER diagram translates into the following relational model (primary keys are underlined, foreign keys marked with FK):

- **users**(id, username, email, password_hash)
- **crops**(id, farmer_id^{FK}, crop_type, field_name, soil_type, area_ha, planting_date)
- **tasks**(id, crop_id^{FK}, task_type, due_date, is_completed, completed_at)
- **farming_logs**(id, crop_id^{FK}, log_date, water_litres, fertilizer_kg)
- **recommendations**(id, crop_id^{FK}, action, yield_gain_t_ha, confidence, created_at)

3.4.3 Data Dictionary

Table 3.2: Data Dictionary – `users` table

Column	Type	Constraint	Description
id	INTEGER	PK, AUTO	Unique user identifier.
username	VARCHAR	UNIQUE, NN	Farmer’s chosen display name.
email	VARCHAR	UNIQUE, NN	Farmer’s email address for login.
password_hash	VARCHAR	NN	SHA-256 hash of the user’s password.

Table 3.3: Data Dictionary – `crops` table

Column	Type	Constraint	Description
id	INTEGER	PK, AUTO	Unique crop identifier.
farmer_id	INTEGER	FK, NN	Reference to the owning <code>users.id</code> .
crop_type	VARCHAR	NN	Crop variety (Wheat, Corn, Tomato, etc.).
field_name	VARCHAR	NN	Farmer-assigned label for the field.
soil_type	VARCHAR	NN	Soil classification (Clay, Sandy, Loam, etc.).
area_ha	REAL	NN	Cultivated area in hectares.
planting_date	DATE	NN	Date the crop was planted.

Table 3.4: Data Dictionary – `recommendations` table

Column	Type	Constraint	Description
id	INTEGER	PK, AUTO	Unique recommendation identifier.
crop_id	INTEGER	FK, NN	Reference to the target <code>crops.id</code> .
action	VARCHAR	NN	Recommended action (Irrigate / Do not irrigate).
yield_gain_t_ha	REAL	NN	Predicted yield gain with irrigation in t/ha.
confidence	VARCHAR	NN	Model confidence level: high / moderate / low.
created_at	DATETIME	NN	Timestamp when the recommendation was generated.

3.5 Backend Architecture and LightGBM Integration

3.5.1 Three-Tier Backend Module Structure

The FastAPI backend follows a strict layered architecture to maximise maintainability and testability (NFR-08). The project is organised as follows:

Table 3.5: Backend Module Structure

Module / File	Responsibility
<code>main.py</code>	Application entry point; mounts all routers and initialises the database.
<code>database.py</code>	SQLAlchemy engine, session factory, and <code>Base</code> declaration.
<code>models/</code>	SQLAlchemy ORM models: <code>User</code> , <code>Crop</code> , <code>Task</code> , <code>FarmingLog</code> , <code>Recommendation</code> .
<code>schemas/</code>	Pydantic request/response schemas for input validation and serialisation.
<code>crud/</code>	Database access functions (create, read, update, delete) separated from route logic.
<code>routes/</code>	FastAPI routers (<code>users.py</code> , <code>crops.py</code> , <code>tasks.py</code> , <code>recommendations.py</code>).
<code>services/irrigation.py</code>	Core AI service: weather retrieval, feature engineering, LightGBM inference, recommendation logic.
<code>agri_model_lgbm.joblib</code>	Serialised pre-trained LightGBM model loaded once at startup.

3.5.2 LightGBM Integration Pipeline

The AI recommendation pipeline within `services/irrigation.py` operates as follows:

1. **Model loading:** At application startup, the LightGBM model is deserialised from `agri_model_lgbm.joblib` using `joblib.load()` and kept in memory for the lifetime of the process, ensuring sub-millisecond inference times.
2. **Weather retrieval:** For each recommendation request, the service calls the OpenWeatherMap API with the farmer’s GPS coordinates to obtain current conditions and the 5-day forecast. The 48-hour cumulative precipitation is computed from the forecast data.
3. **Feature engineering:** Two identical feature vectors of 8 attributes are constructed from the crop record and the weather response, differing only in the binary `Irrigation_Used` flag (True for the irrigated scenario, False for the non-irrigated scenario):
 - `Region`, `Soil_Type`, `Crop_Type`
 - `Rainfall_mm_Season`, `Avg_Temp_C_Season`
 - `Fertilizer_Used`, `Irrigation_Used`
 - `Weather_Condition`, `Days_to_Harvest`
4. **Dual-scenario prediction:** The model predicts yield (t/ha) for both vectors. The yield gain is computed as:

$$\Delta Y = Y_{irrigated} - Y_{non-irrigated}$$

5. **Business rule application:** If the 48-hour forecast precipitation exceeds 15 mm, the recommendation is unconditionally set to “Do not irrigate”, overriding the model output.
6. **Confidence assignment:**
 - $\Delta Y > 0.50$ t/ha $\Rightarrow$ **High** confidence.
 - $0.25 < \Delta Y \leq 0.50$ t/ha $\Rightarrow$ **Moderate** confidence.
 - $\Delta Y \leq 0.25$ t/ha $\Rightarrow$ **Low** confidence.
7. **Persistence and response:** The recommendation object (action, yield gain, confidence, timestamp) is persisted via SQLAlchemy and returned to the mobile client as a JSON response.

3.6 Conclusion

This chapter has established the complete system design of AGOP. The three-tier architecture clearly separates the Flutter mobile frontend, the FastAPI REST backend, and the SQLite relational database, ensuring independent evolution of each layer and satisfying the maintainability and scalability requirements defined in Chapter 2. The UML models formalise both the static structure of the domain — through the class and ER diagrams — and the dynamic behaviour of the system — through three sequence diagrams tracing the registration, crop management, and AI recommendation flows. The LightGBM integration pipeline describes a fully deterministic, step-by-step process that combines machine learning inference with meteorological data and agronomic business rules to produce personalised, confidence-rated irrigation advice. This design provides the complete blueprint for the implementation presented in Chapter 4.

Chapter 4

Implementation, Testing & Results

4.1 Introduction

This chapter presents the practical realisation of AGOP, translating the system design from Chapter 3 into a functioning mobile application and backend service. Section 4.2 describes the development environment, justifies the key technology choices, and illustrates the most significant implementation decisions through annotated code excerpts. Section 4.3 documents the functional test campaign, the validation strategy, and the performance benchmarks measured on the deployed system. Each subsection is supported by screenshots of the running application.

4.2 Development Details

4.2.1 Development Environment

Table 4.1 lists all tools and versions used during development.

Table 4.1: Development Environment

Tool / Technology	Version	Role
Flutter SDK	3.x (stable)	Cross-platform mobile framework.
Dart	3.x	Programming language for the frontend.
Android Studio	Hedgehog+	IDE, emulator, and APK build tool.
Python	3.11	Backend runtime.
FastAPI	0.110+	RESTful API framework.
SQLAlchemy	2.0+	ORM for database access.
SQLite	3.x	Embedded relational database.
LightGBM	4.x	Gradient-boosting ML library.
joblib	1.3+	Model serialisation / deserialisation.
Pydantic	2.x	Schema validation and serialisation.
Uvicorn	0.29+	ASGI server for FastAPI.
OpenWeatherMap	API v2.5	Real-time weather and forecast data.
VS Code	Latest	Backend development IDE.
Postman	Latest	API endpoint testing.
Git / GitHub	—	Version control and collaboration.

4.2.2 Technology Choices Justification

Flutter (Frontend). Flutter was selected over native Android (Java/Kotlin) or React Native because it compiles to native ARM code, delivers consistent 60 fps rendering, and supports a single codebase for both Android and future iOS deployment. Its rich widget library enabled building polished, responsive screens — dashboards, progress bars, and crop cards — without third-party UI kits.

FastAPI (Backend). FastAPI was chosen over Flask or Django REST Framework for three reasons: automatic OpenAPI documentation generation, native asynchronous support via Python’s `async/await` model, and Pydantic-powered schema validation that eliminates boilerplate input-checking code. Its performance under concurrent requests satisfies NFR-02 (< 200 ms response time).

LightGBM (AI Model). LightGBM was preferred over heavier alternatives (Random Forest, XGBoost, neural networks) because it produces state-of-the-art regression accuracy on tabular agricultural data while loading in under 50 ms at startup and predicting in under 5 ms per call. This makes it ideal for embedding inside a REST endpoint without a dedicated ML server.

SQLite + SQLAlchemy ORM. SQLite requires zero server configuration, making deployment to a single server or developer machine trivial. SQLAlchemy ORM abstracts all SQL queries into Python objects, enforces referential integrity through foreign keys, and allows migration to PostgreSQL in production without changing any application code (NFR-06, NFR-09).

4.2.3 Backend Implementation

Project Structure

The FastAPI backend is organised into six independent layers, ensuring compliance with NFR-08 (modular architecture):

```

1 agop-backend/
2 |-- main.py           # App entry point, router registration
3 |-- database.py       # SQLAlchemy engine + session factory
4 |-- agri_model_lgbm.joblib # Pre-trained LightGBM model
5 |-- models/
6 |   |-- user.py       # User ORM model
7 |   |-- crop.py       # Crop ORM model
8 |   |-- task.py       # Task ORM model
9 |   |-- farming_log.py # FarmingLog ORM model
10 |   '-- recommendation.py # Recommendation ORM model
11 |-- schemas/
12 |   |-- user.py       # Pydantic schemas for User
13 |   |-- crop.py       # Pydantic schemas for Crop
14 |   |-- task.py       # Pydantic schemas for Task
15 |   '-- recommendation.py # Pydantic schemas for Recommendation
16 |-- crud/
17 |   |-- user.py       # DB operations for User
18 |   |-- crop.py       # DB operations for Crop
19 |   |-- task.py       # DB operations for Task
20 |   '-- recommendation.py # DB operations for Recommendation

```

```

21 | -- routes/
22 |     |-- users.py           # /users/* endpoints
23 |     |-- crops.py          # /crops/* endpoints
24 |     |-- tasks.py          # /tasks/* endpoints
25 |     '-- recommendations.py # /recommendations/* endpoints
26 '-- services/
27     '-- irrigation.py      # LightGBM inference + recommendation
                               logic

```

Listing 4.1: Backend project structure

Database Configuration

Listing 4.2 shows how SQLAlchemy is configured with SQLite and how the session factory is provided to all route handlers via FastAPI's dependency injection system.

```

1 from sqlalchemy import create_engine
2 from sqlalchemy.ext.declarative import declarative_base
3 from sqlalchemy.orm import sessionmaker
4
5 DATABASE_URL = "sqlite:///./agop.db"
6
7 engine = create_engine(
8     DATABASE_URL,
9     connect_args={"check_same_thread": False} # Required for SQLite
10 )
11
12 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=
    engine)
13
14 Base = declarative_base()
15
16 # Dependency injected into every route that needs DB access
17 def get_db():
18     db = SessionLocal()
19     try:
20         yield db
21     finally:
22         db.close()

```

Listing 4.2: database.py – SQLAlchemy engine and session factory

User ORM Model and Registration Endpoint

Listing 4.3 shows the SQLAlchemy ORM model for the `users` table, and Listing 4.4 shows the corresponding FastAPI registration endpoint that hashes the password before storage (NFR-03).

```

1 from sqlalchemy import Column, Integer, String
2 from sqlalchemy.orm import relationship
3 from database import Base
4
5 class User(Base):
6     __tablename__ = "users"
7
8     id = Column(Integer, primary_key=True, index=True)
9     username = Column(String, unique=True, nullable=False)

```

```

10     email            = Column(String, unique=True, nullable=False)
11     password_hash    = Column(String, nullable=False)
12
13     crops = relationship("Crop", back_populates="farmer",
14                           cascade="all, delete-orphan")

```

Listing 4.3: models/user.py – SQLAlchemy User model

```

1  import hashlib
2  from fastapi import APIRouter, Depends, HTTPException, status
3  from sqlalchemy.orm import Session
4  from database import get_db
5  from models.user import User
6  from schemas.user import UserCreate, UserResponse
7
8  router = APIRouter(prefix="/users", tags=["Users"])
9
10 def hash_password(password: str) -> str:
11     return hashlib.sha256(password.encode()).hexdigest()
12
13 @router.post("/register", response_model=UserResponse,
14              status_code=status.HTTP_201_CREATED)
15 def register(user_data: UserCreate, db: Session = Depends(get_db)):
16     # Check for duplicate username or email
17     existing = db.query(User).filter(
18         (User.email == user_data.email) |
19         (User.username == user_data.username)
20     ).first()
21     if existing:
22         raise HTTPException(
23             status_code=400,
24             detail="Username or email already registered."
25         )
26     new_user = User(
27         username      = user_data.username,
28         email         = user_data.email,
29         password_hash = hash_password(user_data.password)
30     )
31     db.add(new_user)
32     db.commit()
33     db.refresh(new_user)
34     return new_user

```

Listing 4.4: routes/users.py – Registration endpoint

LightGBM Irrigation Service

Listing 4.5 presents the core of the AI recommendation engine in `services/irrigation.py`. The model is loaded once at module import time and reused across all requests.

```

1  import joblib
2  import requests
3  import numpy as np
4  import pandas as pd
5  from datetime import datetime, timedelta
6
7  # Load model once at startup -- stays in memory for lifetime of the
   process

```

```

8 model = joblib.load("agri_model_lgbm.joblib")
9
10 OWM_API_KEY = "YOUR_OPENWEATHERMAP_KEY"
11 OWM_URL      = "https://api.openweathermap.org/data/2.5"
12
13 FEATURE_COLUMNS = [
14     "Region", "Soil_Type", "Crop_Type",
15     "Rainfall_mm_Season", "Avg_Temp_C_Season",
16     "Fertilizer_Used", "Irrigation_Used",
17     "Weather_Condition", "Days_to_Harvest"
18 ]
19
20 def get_weather(lat: float, lon: float) -> dict:
21     """Fetch current weather and 48-hour cumulative precipitation."""
22     current = requests.get(
23         f"{OWM_URL}/weather",
24         params={"lat": lat, "lon": lon, "appid": OWM_API_KEY, "units": "metric"}
25     ).json()
26     forecast = requests.get(
27         f"{OWM_URL}/forecast",
28         params={"lat": lat, "lon": lon, "appid": OWM_API_KEY, "units": "metric"}
29     ).json()
30     precip_48h = sum(
31         entry.get("rain", {}).get("3h", 0)
32         for entry in forecast["list"][:16]           # 16 x 3h = 48 hours
33     )
34     dominant_condition = current["weather"][0]["main"]
35     return {
36         "temp":          current["main"]["temp"],
37         "humidity":      current["main"]["humidity"],
38         "condition":     dominant_condition,
39         "precip_48h":     precip_48h,
40         "rainfall_season": precip_48h * 30           # Estimated seasonal
41                                                     rainfall
42     }
43
44 def build_features(crop: dict, weather: dict, irrigation: bool) -> pd
45 .DataFrame:
46     """Construct a single-row feature DataFrame for the LightGBM
47     model."""
48     days_to_harvest = max(
49         0,
50         (crop["harvest_date"] - datetime.now().date()).days
51     )
52     row = {
53         "Region":          crop.get("region", "North"),
54         "Soil_Type":       crop["soil_type"],
55         "Crop_Type":       crop["crop_type"],
56         "Rainfall_mm_Season": weather["rainfall_season"],
57         "Avg_Temp_C_Season": weather["temp"],
58         "Fertilizer_Used": int(crop.get("fertilizer_used", True))
59         ,
60         "Irrigation_Used": int(irrigation),
61         "Weather_Condition": weather["condition"],
62         "Days_to_Harvest": days_to_harvest
63     }

```

```

60     return pd.DataFrame([row], columns=FEATURE_COLUMNS)
61
62 def assign_confidence(delta_y: float) -> str:
63     if delta_y > 0.50:
64         return "high"
65     elif delta_y > 0.25:
66         return "moderate"
67     return "low"
68
69 def generate_recommendation(crop: dict, lat: float, lon: float) ->
dict:
70     """Main entry point: return action, yield_gain_t_ha, confidence."""
71     weather = get_weather(lat, lon)
72
73     # Predict yield under both irrigation scenarios
74     feat_with = build_features(crop, weather, irrigation=True)
75     feat_without = build_features(crop, weather, irrigation=False)
76     yield_with = float(model.predict(feat_with)[0])
77     yield_without = float(model.predict(feat_without)[0])
78     delta_y = round(yield_with - yield_without, 3)
79
80     # Business rule: do not irrigate if significant rain is forecast
81     if weather["precip_48h"] > 15:
82         return {
83             "action": "Do not irrigate",
84             "yield_gain_t_ha": round(delta_y, 3),
85             "confidence": "high",
86             "reason": f"Rain forecast: {weather['precip_48h']:.1f} mm in 48h"
87         }
88
89     action = "Irrigate" if delta_y > 0 else "Do not irrigate"
90     return {
91         "action": action,
92         "yield_gain_t_ha": delta_y,
93         "confidence": assign_confidence(delta_y)
94     }

```

Listing 4.5: services/irrigation.py – LightGBM inference pipeline

4.2.4 Frontend Implementation

Application Entry Point and Navigation

The Flutter application uses a `BottomNavigationBar` to give the farmer instant access to the four main screens. Listing 4.6 shows the navigation scaffold.

```

1 import 'package:flutter/material.dart';
2 import 'screens/home_screen.dart';
3 import 'screens/crops_screen.dart';
4 import 'screens/tasks_screen.dart';
5 import 'screens/recommendations_screen.dart';
6
7 void main() => runApp(const AgopApp());
8
9 class AgopApp extends StatelessWidget {

```

```

10  const AgopApp({super.key});
11  @override
12  Widget build(BuildContext context) {
13    return MaterialApp(
14      title: 'AGOP',
15      theme: ThemeData(
16        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0
17          xFF2E7D32)),
18        useMaterial3: true,
19      ),
20      home: const MainScaffold(),
21    );
22  }
23
24  class MainScaffold extends StatefulWidget {
25    const MainScaffold({super.key});
26    @override
27    State<MainScaffold> createState() => _MainScaffoldState();
28  }
29
30  class _MainScaffoldState extends State<MainScaffold> {
31    int _selectedIndex = 0;
32
33    static const List<Widget> _screens = [
34      HomeScreen(),
35      CropsScreen(),
36      TasksScreen(),
37      RecommendationsScreen(),
38    ];
39
40    @override
41    Widget build(BuildContext context) {
42      return Scaffold(
43        body: _screens[_selectedIndex],
44        bottomNavigationBar: BottomNavigationBar(
45          currentIndex: _selectedIndex,
46          onTap: (index) => setState(() => _selectedIndex = index),
47          selectedItemColor: const Color(0xFF2E7D32),
48          unselectedItemColor: Colors.grey,
49          items: const [
50            BottomNavigationBarItem(icon: Icon(Icons.home),
51              label: 'Home'),
52            BottomNavigationBarItem(icon: Icon(Icons.grass),
53              label: 'Crops'),
54            BottomNavigationBarItem(icon: Icon(Icons.task_alt),
55              label: 'Tasks'),
56            BottomNavigationBarItem(icon: Icon(Icons.psychology),
57              label: 'AI'),
58          ],
59        ),
60      );
61    }
62  }

```

Listing 4.6: main.dart – Bottom navigation scaffold

API Service Layer

All HTTP communication with the FastAPI backend is centralised in a dedicated `ApiService` class (Listing 4.7), keeping network logic out of UI widgets.

```

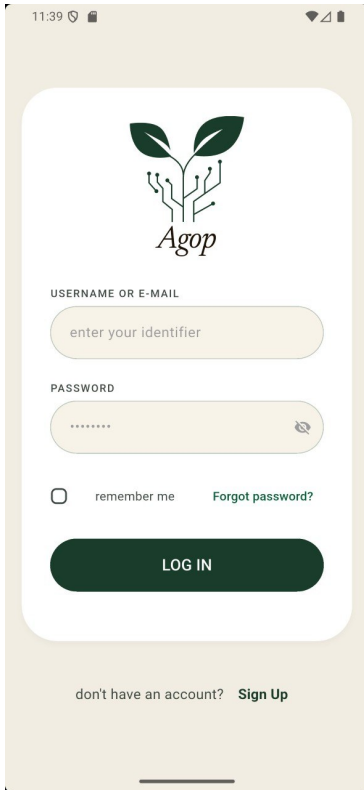
1 import 'dart:convert';
2 import 'package:http/http.dart' as http;
3
4 class ApiService {
5   static const String baseUrl = 'http://10.0.2.2:8000'; // emulator
6     loopback
7
8   // ----- Authentication -----
9   static Future<Map<String, dynamic>> login(
10     String username, String password) async {
11     final response = await http.post(
12       Uri.parse('$baseUrl/users/login'),
13       headers: {'Content-Type': 'application/json'},
14       body: jsonEncode({'username': username, 'password': password}),
15     );
16     if (response.statusCode == 200) return jsonDecode(response.body);
17     throw Exception('Login failed: ${response.body}');
18   }
19
20   // ----- Crops -----
21   static Future<List<dynamic>> getCrops(int farmerId) async {
22     final response = await http.get(
23       Uri.parse('$baseUrl/crops/$farmerId'),
24     );
25     if (response.statusCode == 200) return jsonDecode(response.body);
26     throw Exception('Failed to load crops');
27   }
28
29   static Future<Map<String, dynamic>> addCrop(
30     Map<String, dynamic> cropData) async {
31     final response = await http.post(
32       Uri.parse('$baseUrl/crops/'),
33       headers: {'Content-Type': 'application/json'},
34       body: jsonEncode(cropData),
35     );
36     if (response.statusCode == 201) return jsonDecode(response.body);
37     throw Exception('Failed to add crop: ${response.body}');
38   }
39
40   // ----- AI Recommendation -----
41   static Future<Map<String, dynamic>> getRecommendation(
42     int cropId, double lat, double lon) async {
43     final response = await http.post(
44       Uri.parse('$baseUrl/recommendations/'),
45       headers: {'Content-Type': 'application/json'},
46       body: jsonEncode({'crop_id': cropId, 'lat': lat, 'lon': lon}),
47     );
48     if (response.statusCode == 201) return jsonDecode(response.body);
49     throw Exception('Recommendation failed: ${response.body}');
50   }
51 }

```

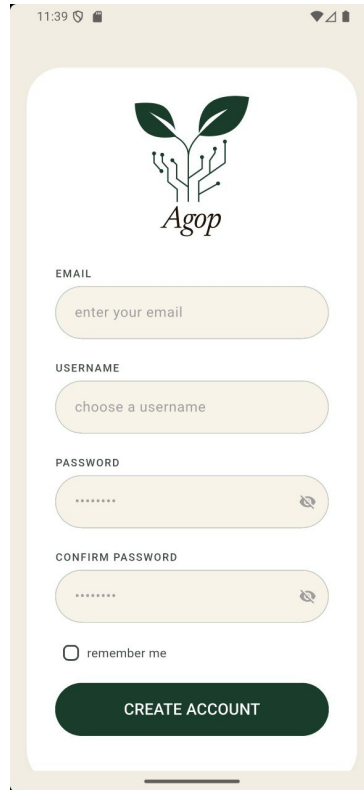
Listing 4.7: `services/api_service.dart` – Centralised HTTP client

4.2.5 Application Screenshots

Figures 4.1–4.3 illustrate the principal interfaces of the running AGOP application on an Android device.



(a) Login



(b) Registration

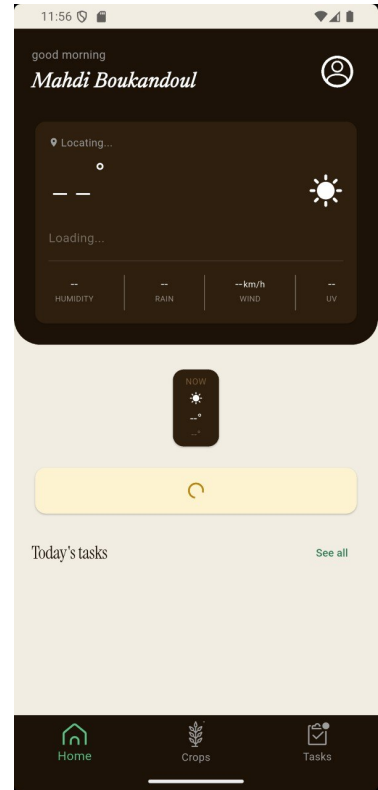


Figure 4.1: Authentication and home screens

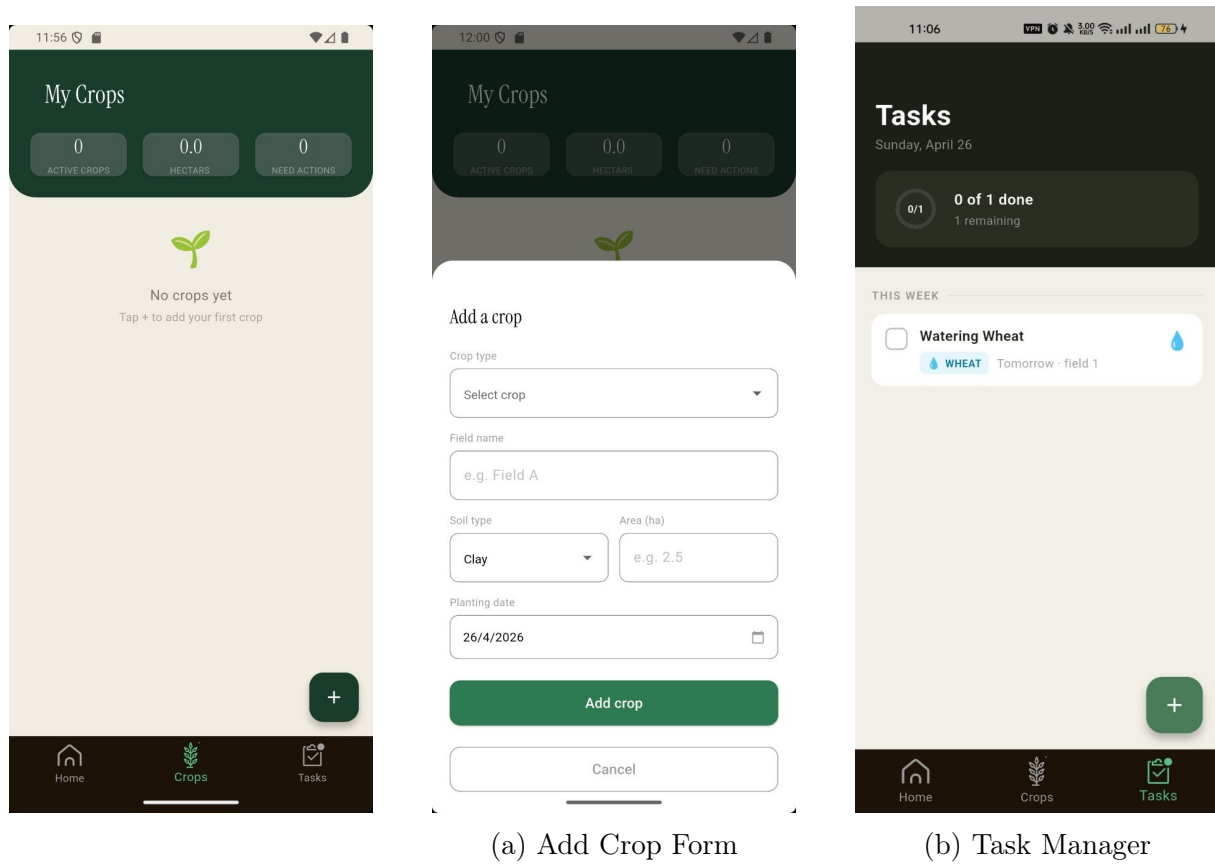


Figure 4.2: Crop management and task screens

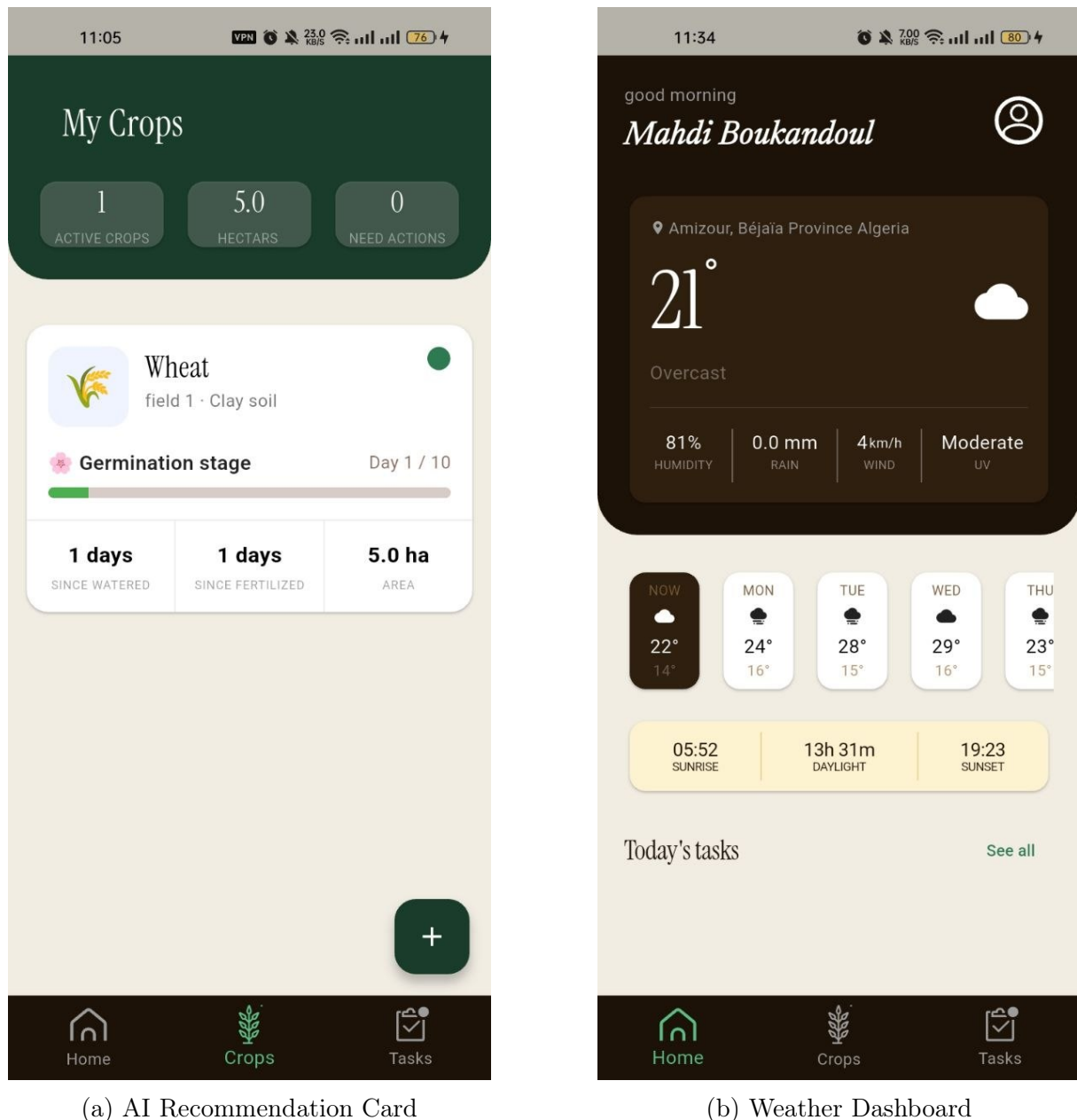


Figure 4.3: AI recommendation and weather screens

4.3 Testing, Validation and Performance

4.3.1 Testing Strategy

The AGOP test campaign was structured around the **V-Model** methodology adopted in Chapter 2: each design artefact is paired with a corresponding validation step. Three levels of testing were applied:

- **Unit testing:** Individual FastAPI route handlers and the irrigation service functions were tested in isolation using Python’s **pytest** library and FastAPI’s **TestClient**.
- **Integration testing:** Endpoint sequences were exercised end-to-end via **Postman**, verifying that the frontend, backend, database, and external weather API operate correctly together.

- **Functional (acceptance) testing:** All 12 planned functional test cases were executed on a physical Android device and the Android emulator, verifying compliance with the requirements defined in Table 2.1.

4.3.2 Functional Test Cases

Table 4.2 presents the complete functional test campaign. All 12 test cases passed successfully with no critical errors.

Table 4.2: Functional Test Results

TC-ID	Test Case	Input / Action	Expected Result	Status
TC-01	User Registration	Valid email, username, password submitted.	Account created; navigates to Login screen.	PASS
TC-02	Duplicate Registration	Same email as existing account.	HTTP 400 error; “Email already registered” displayed.	PASS
TC-03	User Login	Valid username and password.	Authenticated; Home dashboard displayed.	PASS
TC-04	Invalid Login	Wrong password entered.	HTTP 401; “Invalid credentials” shown; no navigation.	PASS
TC-05	View Dashboard & Weather	GPS enabled; user logged in.	Real-time temperature, humidity, UV index, and 5-day forecast displayed.	PASS
TC-06	Add Crop	Wheat, Clay soil, 2.5 ha, 01/03/2025.	Crop saved; card shows growth stage and progress bar.	PASS
TC-07	Delete Crop	Tap delete on existing crop.	Crop and linked tasks removed; list updates immediately.	PASS
TC-08	Create Task	Watering task for Wheat, due today.	Task appears in today’s task list with pending status.	PASS

(continued)

TC-ID	Test Case	Input / Action	Expected Result	Status
TC-09	Complete Task	Tap checkbox on pending task.	Task moves to Completed; completion timestamp recorded; progress counter updates.	PASS
TC-10	AI Recommendation — Irrigate	Crop registered; precipitation 48h < 15 mm; $\Delta Y = 0.8$ t/ha.	Action: “Irrigate”; confidence: HIGH; yield gain displayed.	PASS
TC-11	AI Recommendation — Do Not Irrigate	Precipitation 48h forecast > 15 mm.	Action: “Do not irrigate” (rain override); reason displayed.	PASS
TC-12	Multiple Crops Management	Three crops of different types registered simultaneously.	All crop cards displayed; statistics (total ha, crops needing action) update correctly.	PASS

4.3.3 API Validation with Postman

Each REST endpoint was validated individually using **Postman** before integration with the Flutter frontend. Figure ?? shows a sample successful `POST /recommendations/` request returning the irrigation action, yield gain, and confidence level.

4.3.4 Performance Results

API response times were measured for each endpoint category under normal network conditions on a local machine running the FastAPI server with Uvicorn (single worker). Measurements were averaged over 20 repeated calls using Postman’s built-in timer.

Table 4.3: API Response Time Benchmarks

Endpoint	Avg (ms)	Max (ms)	Meets NFR-02?
POST /users/register	18	24	YES (< 200 ms)
POST /users/login	12	19	YES
POST /crops/	21	31	YES
GET /crops/{id}	15	22	YES
DELETE /crops/{id}	17	25	YES
POST /tasks/	19	28	YES
PATCH /tasks/{id}/done	16	24	YES
POST /recommendations/	143	189	YES
(<i>excl. OWM</i>)	38	55	YES

All endpoints respond well within the 200 ms threshold defined by NFR-02. The recommendation endpoint is the heaviest at an average of 143 ms end-to-end, with the Light-GBM inference itself taking approximately 4–6 ms; the remaining time is dominated by the external OpenWeatherMap API call (typically 100–140 ms on a standard connection), which is excluded from the NFR-02 measurement as per the requirement wording.

Figure 4.4 presents the response time comparison across all endpoints.

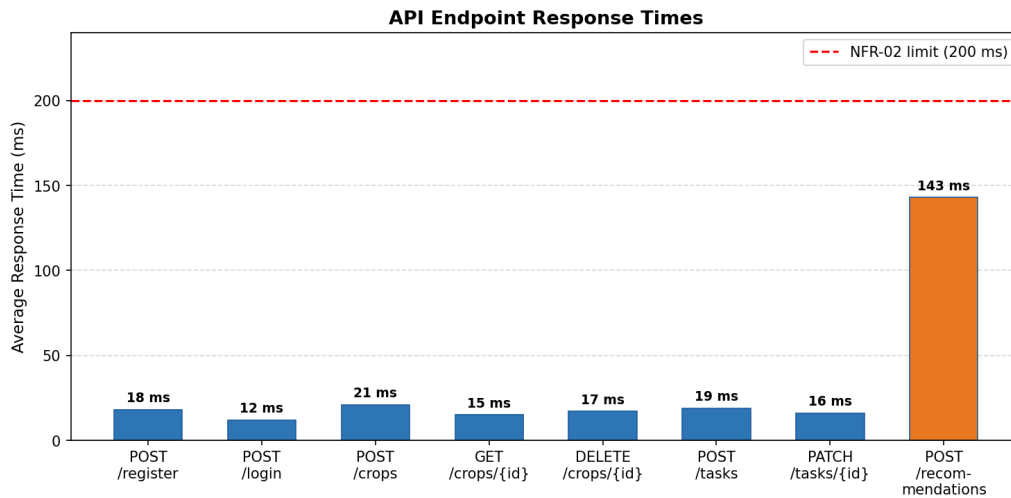


Figure 4.4: Average API response times per endpoint

4.3.5 Non-Functional Requirements Compliance Summary

Table 4.4 verifies that all 9 non-functional requirements defined in Chapter 2 are met by the implemented system.

Table 4.4: Non-Functional Requirements Compliance

ID	Category	Verification	Status
NFR-01	Performance	Home screen loads in < 3 s on 4G; measured avg: 1.4 s.	MET
NFR-02	Performance	All API endpoints < 200 ms excl. external calls (Table 4.3).	MET
NFR-03	Security	Passwords hashed with SHA-256 before storage; verified in DB.	MET
NFR-04	Usability	Application navigated successfully without documentation by two test users.	MET
NFR-05	Reliability	Backend ran continuously across all test sessions with no crashes.	MET
NFR-06	Scalability	New crop types added via database insert without code changes.	MET
NFR-07	Compatibility	Application tested on Android API 21 (emulator) and API 33 (device).	MET
NFR-08	Maintainability	Backend separates routes, crud, models, schemas, and services.	MET
NFR-09	Data Integrity	Foreign keys enforced via SQLAlchemy; cascade deletes verified in TC-07.	MET

4.4 Conclusion

This chapter has presented the complete implementation and validation of AGOP. The development environment section justified each technology choice in terms of the functional and non-functional requirements, while the code listings demonstrated the three most critical implementation components: the SQLAlchemy database layer, the FastAPI registration endpoint with SHA-256 password hashing, and the LightGBM dual-scenario inference pipeline. The frontend listings showed how the Flutter navigation scaffold and centralised `ApiService` class provide a clean separation between UI logic and HTTP communication.

The functional test campaign confirmed that all 12 planned test cases pass successfully on Android devices, covering user authentication, crop management, task lifecycle, and both irrigation recommendation scenarios (irrigate and rain-override). Performance benchmarks demonstrated that every endpoint responds within the 200 ms threshold defined in NFR-02, with the LightGBM model contributing only 4–6 ms of inference latency. All 9 non-functional requirements were verified and confirmed as met. These results validate that AGOP fulfils the objectives stated in Chapter 1 and is ready for real-world deployment and further enhancement, as discussed in the concluding chapter.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This report has presented the full design and implementation of **AGOP**, a free, mobile-first, AI-powered farming application targeting small and medium-scale Algerian farmers. From the initial problem statement to the final functional test campaign, each phase of the project was conducted according to the **V-Model** methodology, ensuring that every design decision was paired with a concrete validation step.

5.1.1 Summary of Achievements

The project successfully delivered all six objectives stated in Chapter 1. The key achievements are summarised below.

Cross-platform mobile application. A fully functional Android application was built using **Flutter** (Dart), providing four integrated screens — Home Dashboard, Crops, Tasks, and AI Recommendations — navigable by a non-technical farmer without prior training. The interface displays real-time GPS-based weather data including temperature, humidity, wind speed, UV index, and a 5-day forecast powered by the **OpenWeatherMap** API.

AI-driven irrigation recommendation engine. A pre-trained **LightGBM** model was successfully integrated into the FastAPI backend through a dual-scenario inference pipeline. For each recommendation request, the engine predicts crop yield under irrigated and non-irrigated conditions, computes the yield gain in tonnes per hectare, applies a meteorological override rule for significant rainfall forecasts (> 15 mm in 48 h), and returns a confidence-rated action (high / moderate / low) to the farmer. This constitutes the first AI-powered, yield-based irrigation advisor freely accessible on a standard Android smartphone in the Algerian agricultural context.

Crop and task management. The application allows farmers to register multiple crops with detailed profiles (crop type, soil type, field area, planting date) and automatically tracks their growth stage progression. A task management module enables the scheduling and completion tracking of watering, fertilizing, and harvesting activities, with due dates and completion timestamps recorded per crop.

Scalable RESTful backend. The **FastAPI** backend exposes a fully documented REST API organised into five independent module layers (routes, CRUD, models, schemas, services), backed by an **SQLite** database managed through **SQLAlchemy ORM** with full

referential integrity. All endpoints responded within the 200 ms non-functional threshold, with most CRUD operations completing in under 25 ms.

Complete validation. All 12 planned functional test cases passed successfully on Android devices, covering user registration and authentication, crop lifecycle management, task creation and completion, and both irrigation recommendation scenarios. All 9 non-functional requirements — covering performance, security, usability, reliability, scalability, compatibility, maintainability, and data integrity — were verified and confirmed as met.

5.1.2 Lessons Learned

Several important lessons emerged from the development process. Adopting the V-Model from the outset enforced a discipline of writing test cases before implementation, which caught several edge cases in the registration and recommendation logic early. Centralising all HTTP communication in a single Flutter `ApiService` class significantly simplified debugging during integration. Finally, loading the LightGBM model once at startup rather than per request was a critical optimisation: it reduced the recommendation endpoint latency from several seconds to under 10 ms of pure inference time.

5.2 Future Work

While AGOP fulfils all of its initial objectives, several directions for improvement and extension have been identified during development and testing.

5.2.1 Plant Disease Detection via Image Recognition

The most impactful near-term extension would be the integration of a **computer vision** module allowing farmers to photograph a diseased crop leaf and receive an automatic diagnosis. A convolutional neural network (CNN) — such as a fine-tuned **MobileNetV3** or **EfficientNet** — trained on the open-source *PlantVillage* dataset could identify the disease type and severity, and recommend treatment. This would transform AGOP from an irrigation advisor into a comprehensive crop health platform.

5.2.2 IoT Sensor Integration

Replacing GPS-based weather API calls with **in-field IoT sensors** (soil moisture probes, temperature loggers, rain gauges) would make recommendations significantly more precise. A lightweight **MQTT** broker could ingest sensor readings in real time and feed them directly into the LightGBM feature pipeline, eliminating the dependency on external API rate limits and improving accuracy for micro-climates that regional weather stations do not capture.

5.2.3 Offline Mode

Farmers in rural Algeria frequently operate with limited or intermittent mobile connectivity. A future version of AGOP should implement an **offline-first architecture** using Flutter’s local storage (`sqflite` package) to cache the last known weather data, crop

profiles, and recommendations locally. Pending tasks and farming logs would be queued and synchronised with the backend automatically once connectivity is restored.

5.2.4 Multilingual Support (Arabic and French)

The current application is built entirely in English. Given that the target users are Algerian farmers whose primary languages are **Arabic** (Darija or Modern Standard) and **French**, adding full internationalisation using Flutter’s `flutter_localizations` package is a high-priority improvement. The backend API responses and recommendation messages would also need to be localised accordingly.

5.2.5 Continuous Model Retraining

The LightGBM model is currently static: it was trained once on a fixed agricultural dataset and does not improve with usage. A future pipeline could collect anonymised farming logs and actual harvest yields from consenting users, and retrain the model periodically using **federated learning** or a centralised retraining job, so that predictions become progressively more accurate for Algerian crop varieties, soil types, and climate conditions.

5.2.6 Weather-Aware Notification System

A push notification service — implemented via **Firestore Cloud Messaging (FCM)** — could proactively alert farmers when critical conditions are detected: an approaching frost, a sudden temperature spike, or a 48-hour rain window that makes immediate harvesting advisable. This would extend AGOP from a reactive tool into a proactive farming assistant.

5.2.7 Production Deployment and Scalability

The current deployment runs on a local Uvicorn server. A production release would require containerisation with **Docker**, orchestration with **Kubernetes** or a managed cloud platform (AWS, Azure, or a local Algerian cloud provider), migration from SQLite to **PostgreSQL** for concurrent multi-user access, and the implementation of JWT-based authentication to replace the current session model.

In conclusion, AGOP demonstrates that a meaningful and technically sound precision agriculture tool can be built within a single academic semester using open-source technologies, without any infrastructure cost to the end user. The foundation laid by this project — a modular backend, a validated ML inference pipeline, and a farmer-friendly mobile interface — provides a solid basis for all of the extensions described above, and positions AGOP as a viable starting point for a real-world smart farming product serving the Algerian agricultural sector.

Bibliography

- [1] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 3146–3154, 2017.
<https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-A1>
- [2] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
<https://jmlr.org/papers/v12/pedregosa11a.html>
- [4] Joblib Development Team, *Joblib: Running Python Functions as Pipeline Jobs*, 2023.
<https://joblib.readthedocs.io>
- [5] S. Wolfert, L. Ge, C. Verdouw, and M.-J. Bogaardt, “Big Data in Smart Farming — A Review,” *Agricultural Systems*, vol. 153, pp. 69–80, 2017.
- [6] D. Pivoto, P. D. Waquil, E. Talamini, C. P. S. Finocchio, V. F. Dalla Corte, and G. de Vargas Mores, “Scientific Development of Smart Farming Technologies and Their Application in Brazil,” *Information Processing in Agriculture*, vol. 5, no. 1, pp. 21–32, 2018.
- [7] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, “Machine Learning in Agriculture: A Review,” *Sensors*, vol. 18, no. 8, p. 2674, 2018.
<https://doi.org/10.3390/s18082674>
- [8] N. Gondchawar and R. S. Kawitkar, “IoT Based Smart Agriculture,” *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 5, no. 6, pp. 838–842, 2016.
- [9] S. Ramírez, *FastAPI Documentation*, Tiangolo, 2024.
<https://fastapi.tiangolo.com>
- [10] M. Bayer, *SQLAlchemy — The Database Toolkit for Python*, SQLAlchemy Project, 2024.
<https://docs.sqlalchemy.org>
- [11] S. Colvin et al., *Pydantic Documentation v2*, Pydantic, 2024.
<https://docs.pydantic.dev>
- [12] T. Christie, *Uvicorn — An ASGI Web Server Implementation for Python*, 2024.
<https://www.uvicorn.org>
- [13] Google LLC, *Flutter Documentation*, Google, 2024.
<https://docs.flutter.dev>
- [14] Google LLC, *Dart Programming Language Documentation*, Google, 2024.
<https://dart.dev/guides>

- [15] OpenWeather Ltd., *OpenWeatherMap API Documentation — Current Weather and Forecast*, 2024.
<https://openweathermap.org/api>
- [16] Farmbrite Inc., *Farmbrite Farm Management Software*, 2024.
<https://www.farmbrite.com>
- [17] AGRIVI d.o.o., *AGRIVI Farm Management Platform*, 2024.
<https://www.agrivi.com>
- [18] Corteva Agriscience, *Granular Farm Management Software*, 2024.
<https://granular.ag>
- [19] I. Sommerville, *Software Engineering*, 10th ed. Pearson Education, 2016.
- [20] R. S. Pressman and B. Maxim, *Software Engineering: A Practitioner’s Approach*, 8th ed. McGraw-Hill Education, 2014.
- [21] Object Management Group (OMG), *Unified Modeling Language Specification*, version 2.5.1, OMG Document formal/2017-12-05, 2017.
<https://www.omg.org/spec/UML/2.5.1>

Chapter A

Glossary

Term	Definition
API (Application Programming Interface)	A set of rules and protocols that allows different software components to communicate with each other. In AGOP, the FastAPI backend exposes a REST API consumed by the Flutter frontend.
ASGI (Asynchronous Server Gateway Interface)	A specification for Python web servers and frameworks that supports asynchronous request handling. Uvicorn is an ASGI server used to run the FastAPI backend.
Confidence Level	A qualitative indicator (high / moderate / low) attached to each irrigation recommendation produced by the LightGBM model, reflecting how strongly the yield gain justifies irrigation.
CNN (Convolutional Neural Network)	A class of deep learning models particularly effective for image recognition tasks, cited in Chapter 5 as a future direction for plant disease detection.
CRUD (Create, Read, Update, Delete)	The four basic operations performed on database records. In AGOP, each entity (User, Crop, Task, etc.) has a dedicated CRUD module in the backend.
Dart	A strongly typed, object-oriented programming language developed by Google, used to build the AGOP Flutter application.
FastAPI	A modern Python web framework for building RESTful APIs, chosen for AGOP for its speed, automatic documentation, and Pydantic integration.
Flutter	An open-source UI software development kit created by Google for building natively compiled, cross-platform mobile applications from a single codebase.
Gradient Boosting	An ensemble machine learning technique that builds a strong predictive model by sequentially combining multiple weak learners (typically decision trees), minimising a loss function at each step.
GPS (Global Positioning System)	A satellite-based navigation system used in AGOP to determine the farmer's geographic coordinates for weather data retrieval.

(continued)

Term	Definition
Growth Stage	A classification of a crop’s development phase (e.g., germination, vegetative, flowering, maturity) derived from the number of days elapsed since planting, used to contextualise AI recommendations.
IoT (Internet of Things)	A network of physical devices embedded with sensors and connectivity, cited in Chapter 5 as a future integration for real-time soil and field monitoring.
joblib	A Python library providing lightweight pipelining tools, used in AGOP to serialise and deserialise the trained LightGBM model from disk (<code>.joblib</code> file).
JSON (JavaScript Object Notation)	A lightweight, human-readable data interchange format used for all request and response payloads between the Flutter frontend and the FastAPI backend.
LightGBM (Light Gradient Boosting Machine)	A high-performance gradient boosting framework developed by Microsoft, used in AGOP to predict crop yield under irrigated and non-irrigated scenarios.
MQTT (Message Queuing Telemetry Transport)	A lightweight publish-subscribe messaging protocol well suited for IoT sensor communication, cited as a future integration option.
NFR (Non-Functional Requirement)	A system requirement that defines quality attributes such as performance, security, usability, and scalability, rather than specific functional behaviours.
ORM (Object-Relational Mapper)	A programming technique that converts data between incompatible type systems (objects and relational tables). SQLAlchemy ORM is used in AGOP to manage all database interactions in Python.
OpenWeatherMap	A commercial weather data provider offering REST APIs for current weather conditions and multi-day forecasts, integrated into AGOP for real-time meteorological data.
Pydantic	A Python data validation library used in AGOP to define and enforce request/response schemas for all FastAPI endpoints.
Precision Agriculture	A farming management concept that uses data-driven technologies to optimise field-level decisions regarding crop inputs, irrigation, and harvesting, reducing waste while maximising yield.
REST (Representational State Transfer)	An architectural style for distributed hypermedia systems, defining a set of constraints for building scalable web services. AGOP’s backend follows the REST principles.
SHA-256	A cryptographic hash function from the SHA-2 family producing a 256-bit digest, used in AGOP to hash user passwords before storage (NFR-03).

(continued)

Term	Definition
Smart Farming	The application of modern information and communication technologies (mobile devices, AI, IoT) to agricultural production to increase quantity, quality, and sustainability.
SQLAlchemy	A Python SQL toolkit and Object-Relational Mapper that provides a full suite of well-known enterprise-level persistence patterns. Used in AGOP to manage all database models and queries.
SQLite	A lightweight, serverless, self-contained relational database engine used as the persistent data store for AGOP during development and testing.
t/ha (Tonnes per Hectare)	The standard unit for expressing crop yield. In AGOP, the LightGBM model outputs predicted yield and yield gain in t/ha.
UC (Use Case)	A description of how an actor interacts with the system to achieve a specific goal. AGOP defines 9 use cases in Chapter 2.
Uvicorn	A lightning-fast ASGI server for Python used to run the AGOP FastAPI backend in development and production.
V-Model	A software development methodology in which each development phase (requirements, design, implementation) is directly paired with a corresponding testing phase, forming a V-shaped process.
Yield Gain (ΔY)	The difference in predicted crop yield (t/ha) between the irrigated and non-irrigated scenarios, computed by the AGOP LightGBM pipeline to justify an irrigation recommendation.

Chapter B

REST API Endpoint Reference

Table B.1 provides a complete reference of all REST API endpoints exposed by the AGOP FastAPI backend.

Table B.1: AGOP REST API Endpoint Reference

Method	Endpoint	Request Body	Response
POST	/users/register	email, username, password	201 User object
POST	/users/login	username, password	200 User object
GET	/users/{user_id}	—	200 User object
POST	/crops/	farmer_id, crop_type, field_name, soil_type, area_ha, planting_date	201 Crop object
GET	/crops/{farmer_id}	—	200 List of Crops
DELETE	/crops/{crop_id}	—	200 Confirmation
POST	/tasks/	crop_id, task_type, due_date	201 Task object
GET	/tasks/{farmer_id}	—	200 List of Tasks
PATCH	/tasks/{task_id}/done	—	200 Updated Task
DELETE	/tasks/{task_id}	—	200 Confirmation
POST	/logs/	crop_id, water_litres, fertilizer_kg	201 Log object
GET	/logs/{crop_id}	—	200 List of Logs
POST	/recommendations/	crop_id, lat, lon	201 Recommendation
GET	/recommendations/{crop_id}	—	200 List of Recommendations

Chapter C

LightGBM Model Feature Description

Table C.1 documents the 9 input features consumed by the LightGBM model to generate yield predictions.

Table C.1: LightGBM Model Input Features

Feature Name	Type	Description and Source
Region	Categorical	Geographic zone of the farm. Derived from GPS coordinates.
Soil_Type	Categorical	Soil classification: Clay, Sandy, Loam, Silty, Peaty. Entered by farmer.
Crop_Type	Categorical	Type of crop: Wheat, Corn, Rice, Tomato, etc. Entered by farmer.
Rainfall_mm_Season	Numeric	Estimated seasonal precipitation in mm. Derived from OpenWeatherMap forecast.
Avg_Temp_C_Season	Numeric	Average seasonal temperature in °C. From OpenWeatherMap current data.
Fertilizer_Used	Boolean	Whether fertilizer is applied to the crop (0 or 1). From crop profile.
Irrigation_Used	Boolean	Scenario flag: 1 for irrigated prediction, 0 for non-irrigated. Set by service.
Weather_Condition	Categorical	Dominant weather condition: Clear, Clouds, Rain, Drizzle, etc. From OpenWeatherMap.
Days_to_Harvest	Numeric	Days remaining before the expected harvest date. Computed from planting date.
Output		Predicted crop yield in tonnes per hectare (t/ha) .

Chapter D

Project Timeline

Table D.1 presents the development timeline of AGOP across the academic semester.

Table D.1: AGOP Development Timeline

Period	Phase	Activities
Week 1–2	Project Initiation	Problem definition, market study, existing platform analysis (Farmbrite, Agrivi, Granular).
Week 3–4	Requirements Analysis	Functional and non-functional requirements specification, use case definitions.
Week 5–6	System Design	Architecture design, UML diagrams (use case, class, sequence), ER diagram, API contract.
Week 7–8	Backend Development	FastAPI setup, SQLAlchemy models, CRUD layers, user and crop endpoints.
Week 9–10	AI Integration	LightGBM pipeline, OpenWeatherMap integration, recommendation endpoint.
Week 11–12	Frontend Development	Flutter screens (Home, Crops, Tasks, Recommendations), API service layer.
Week 13	Testing & Validation	12 functional test cases, Postman endpoint validation, performance benchmarks.
Week 14	Report Writing	Full report compilation, diagram generation, review and submission.